

SOLIDWORKS API (.NET, VBA, C#) — Examples (21/50): Fire_Notification_After_Adding_a_Mate_Example Get_All_Sheet_Metal_Feature_Data_Example

SOLIDWORKS API Documentation (converted from CHM)

Fire Notification After Adding a Mate Example (VB.NET)

This example shows how to fire a post-notify event when adding mates.

```

-----
Preconditions:
1. Verify that the specified assembly to open exists.
2. Click Tools > Options > clear the Stop VSTA debugger on macro exit check box if it is selected.
3. Open the Immediate window.

Postconditions:
1. Opens the assembly.
2. Interactively add a mate between two entities (Click Insert > Mate). For example, add a lock mate between the toaster base and the cup.
3. Adds a mate between the selected entities.
4. Examine the Immediate window.
5. Click Debug > Stop Debugging in SOLIDWORKS Visual Studio Tools for Applications.
6. Click Tools > Options > Stop VSTA debugger on macro exit check box in SOLIDWORKS.

NOTE: Because this assembly is used elsewhere, do not save changes.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

```

Partial Class SolidWorksMacro

```

    Public WithEvents swAssemblyDoc As AssemblyDoc

    Public Sub Main()
        Dim swModel As ModelDoc2

        Dim errorstatus As Long, warningstatus As Long
        Dim openAssem As Hashtable

        ' Open assembly document
        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2019\samples\tutorial\api\toaster_scene.sldasm", swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errorstatus, warningstat
us)

        swModel = swApp.ActiveDoc

        ' Set up event
        swAssemblyDoc = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssemblyDoc.AddMatePostNotify2, AddressOf Me.swAssemblyDoc_AddMatePostNotify2
    End Sub

    Private Function swAssemblyDoc_AddMatePostNotify2(ByRef mates As Object) As Integer
        Debug.Print("One or more mates were added to the assembly.")
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Fire Notifications for Background Processing Events Example (C#)

This example shows how to fire notifications when background processing events occur.

```
//-----
// Preconditions:
// 1. Create a VSTA C# macro.
//   a. Copy and paste SolidWorksMacro.cs code in the macro.
//   b. Create a form, Form1, that contains the following
//      controls:
//      * CheckBox1 with caption Enable background processing and open
//        drawing.
//      * button1 with caption Close after background processing end event
//        fires".
//   c. Copy and paste Form1.cs code in your form's code window.
//   d. Modify the path in Form1.cs to open a huge drawing document that
//      contains many parts.
// 2. Press F5 to start and close the debugger.
// 3. Click Build > Build macro_name to build a DLL for the macro.
// 4. Save and close the macro.
//
// Postconditions:
// 1. Open the Windows Task manager, click the Processes tab, and click the CPU column
//    header to sort the processes in descending order.
// 2. In SOLIDWORKS, click Tools > Macro > Run.
//   a. Locate your macro's \SwMacro\bin\Debug folder.
//   b. Select macro_name.dll.
//   c. Click Open to open the form.
// 3. Select the Enable background processing and open drawing checkbox on the form.
// 4. Displays a checkmark in the check box.
// 5. Click OK to close the Background processing enabled message box.
// 6. Opens the specified drawing.
// 7. Fires the background processing start events.
// 8. Click OK to close the Background processing start event fired message box.
// 9. In the Windows Task Manager, observe that several sldbgproc.exe processes are
//    occupying most of the CPU.
// 10. Click OK to close the Background processing stop event fired message box.
// 11. Click Close after background processing end event fired button on the form.
// 12. Unloads Form1.
//-----
```

//SolidWorksMacro.cs

```
using SolidWorks.Interop.sldworks;
using System.Runtime.InteropServices;
using System;
using System.Windows.Forms;

namespace BackgroundProcessingEventsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public SldWorks swApp;

        public void Main()
        {
            //Create and show an instance of the form
            Form1 myForm = new Form1();
            myForm.Show();
        }
    }
}
```

//Form1

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Windows.Forms;
using System.Diagnostics;
using System.Collections;
using System.Runtime.InteropServices;
using System;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;

namespace BackgroundProcessingEventsCSharp.csproj
{
    public partial class Form1 : Form
    {
        public Form1()
        {

```

```

        InitializeComponent();
    }

    public SldWorks swApp;
    public bool checkBoxClicked;

    private void checkBox1_CheckedChanged(object sender, EventArgs e)
    {
        try
        {
            swApp = (SldWorks)System.Runtime.InteropServices.Marshal.GetActiveObject("SldWorks.Application");
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
            return;
        }

        ModelDoc2 swModelDoc = default(ModelDoc2);
        DrawingDoc swDrawingDoc = default(DrawingDoc);

        string filePath = null;
        filePath = "path_and_filename_of_huge_drawing";

        DocumentSpecification docSpecification = default(DocumentSpecification);

        // Set up events
        AttachEventHandlers();

        // Enable background processing
        swApp.EnableBackgroundProcessing = true;
        MessageBox.Show("Background processing enabled");

        // Open huge drawing
        docSpecification = (DocumentSpecification)swApp.GetOpenDocSpec(filePath);
        docSpecification.Silent = true;
        swModelDoc = (ModelDoc2)swApp.OpenDoc7(docSpecification);

        swDrawingDoc = (DrawingDoc)swModelDoc;

        // Set document background processing to application setting
        swDrawingDoc.BackgroundProcessingOption = (int)swBackgroundProcessOption_e.swBackgroundProcessing_DeferToApplication;
    }

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        swApp.BackgroundProcessingStartNotify += this.mySwApp_BackgroundProcessingStartNotify;
        swApp.BackgroundProcessingEndNotify += this.mySwApp_BackgroundProcessingEndNotify;
    }

    private int mySwApp_BackgroundProcessingStartNotify(string filename)
    {
        MessageBox.Show("Background processing start event fired");
        return 1;
    }

    private int mySwApp_BackgroundProcessingEndNotify(string filename)
    {
        MessageBox.Show("Background processing end event fired");
        swApp.EnableBackgroundProcessing = false;
        return 1;
    }

    public void CheckBox1_Click(object sender, System.EventArgs e)
    {
        checkBoxClicked = true;
    }

    private void button1_Click(object sender, EventArgs e)
    {
        this.Close();
    }
}
}

```

Fire Notifications for Background Processing Events Example (VBA)

This example shows how to fire notifications when background processing events occur.

```

'-----
' Preconditions:
' 1. Create a form, UserForm1, that contains the following controls:
'   * CheckBox1 with caption Enable background processing and open drawing
'   * CommandButton1 with caption Close after background processing end event fires
' 2. Copy this code to the main module.
' 3. Copy this code to UserForm1.
' 4. Modify the path in UserForm1's code to point to a large drawing document
'   containing many parts.
' 5. Open the Task Manager, click the Processes tab, and click the
'   CPU column header to sort the processes in descending order.
' 6. Run this macro to show UserForm1.
'   a. Select Enable background processing and open drawing.
'       1. Enables background processing for the application.
'       2. Opens the specified drawing document.
'       3. Fires background processing start event.
'   b. In Task Manager, observe that several sldbgproc.exe processes
'       are occupying most of the CPU.
'   c. After several seconds, observe that the sldbgproc.exe processes stop,
'       and the background processing end event fires.
'   d. Click Close after background processing end event fires.
'       UserForm1 unloads.
'
' Postconditions: Inspect the Immediate window for background processing
' event notifications.
'-----

'Main module
Option Explicit

Sub main()
    UserForm1.Show vbModeless
End Sub

'UserForm1 Code

Option Explicit
Private WithEvents swApp As SldWorks.SldWorks
Private Sub CheckBox1_Click()
    Set swApp = Application.SldWorks
    swApp.EnableBackgroundProcessing = CheckBox1.Value

    Dim filePath As String
    filePath = "path_to_large_drawing"
    Dim docSpecification As DocumentSpecification
    Set docSpecification = swApp.GetOpenDocSpec(filePath)
    docSpecification.Silent = True

    Dim swmodelDoc As ModelDoc2
    Set swmodelDoc = swApp.OpenDoc7(docSpecification)
    Dim swDrawingDoc As DrawingDoc
    Set swDrawingDoc = swmodelDoc

    ' Set document background processing to application setting
    swDrawingDoc.BackgroundProcessingOption = 2
End Sub

Private Function swApp_BackgroundProcessingStartNotify(ByVal filename As String) As Long
    Debug.Print "Background processing start event fired."
End Function

Private Function swApp_BackgroundProcessingEndNotify(ByVal filename As String) As Long
    Debug.Print "Background processing end event fired."
    swApp.EnableBackgroundProcessing = False
End Function

Private Sub CommandButton1_Click()
    Unload UserForm1
End Sub

```

Fire Notifications for Background Processing Events Example (VB.NET)

This example shows how to fire notifications when background processing events occur.

```

Preconditions:
1. Create a VSTA VB.NET macro.
  a. Copy and paste SolidWorksMacro.vb's code in the macro.
  b. Create a form, Form1, that contains the following controls:
      * CheckBox1 with caption Enable background processing and open drawing.
      * Button1 with caption Close after background processing end event fires.
  c. Copy and paste Form1.vb's code in your form's code window.
  d. Modify the path in Form1.vb to open a huge drawing document that contains many parts.
2. Press F5 to start and close the debugger.
3. Click Build > Build macro_name to build a DLL for the macro.
4. Save and close the macro.

Postconditions:
1. Open the Windows Task manager, click the Processes tab, and click the CPU column header to sort the processes in descending order.
2. In SOLIDWORKS, click Tools > Macro > Run.
  a. Locate your macro's \SwMacro\bin folder.
  b. Select macro_name.dll.
  c. Click Open to open the form.
3. Select the Enable background processing and open drawing checkbox on the form.
4. Displays a checkmark in the check box.
5. Click OK to close the Background processing enabled message box.
6. Opens the specified drawing.
7. Fires the background processing start events.
8. Click OK to close the Background processing start event fired message box.
9. In the Windows Task Manager, observe that several sldbgproc.exe processes are occupying most of the CPU.
10. Click OK to close the Background processing end event fired message box.
11. Click Close after background processing end event fires button on the form.
12. Unloads Form1.

```

'SolidWorksMacro.vb

```

Imports SolidWorks.Interop.sldworks
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Public Class SolidWorksMacro
```

```
    Public swApp As SldWorks
```

```
    Public Sub Main()
```

```
        'Create and show an instance of the form
        Dim myForm As New Form1
        myForm.Show()
    End Sub
End Class
```

```
'Form1
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Windows.Forms
Imports System.Diagnostics
Imports System.Collections
Imports System.Runtime.InteropServices

```

```
Public Class Form1
```

```
    Public WithEvents swApp As SldWorks
    Public checkBoxClicked As Boolean

```

```
    Private Sub CheckBox1_CheckedChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles CheckBox1.CheckedChanged
```

```
        Try
            swApp = CType(System.Runtime.InteropServices.Marshal.GetObject("SldWorks.Application"), SldWorks)
        Catch ex As Exception
            MsgBox(ex.Message)
        Exit Sub
    End Try

```

```
    Dim swModelDoc As ModelDoc2
    Dim swDrawingDoc As DrawingDoc

```

```
    Dim filePath As String
    filePath = "Path_and_filename_of_huge_drawing"

```

```
    Dim docSpecification As DocumentSpecification

```

```
    ' Set up events
    AttachEventHandlers()

```

```
    ' Enable background processing
    swApp.EnableBackgroundProcessing = True
    MsgBox("Background processing enabled")

```

```
    ' Open huge drawing
    docSpecification = swApp.GetOpenDocSpec(filePath)
    docSpecification.Silent = True
    swModelDoc = swApp.OpenDoc7(docSpecification)

```

```
    swDrawingDoc = swModelDoc

```

```
    ' Set document background processing to application setting
    swDrawingDoc.BackgroundProcessingOption = swBackgroundProcessOption_e.swBackgroundProcessing_DeferToApplication

```

```

End Sub

Sub AttachEventHandlers()
    AttachSWEvents()
End Sub

Sub AttachSWEvents()
    AddHandler swApp.BackgroundProcessingStartNotify, AddressOf Me.mySwApp_BackgroundProcessingStartNotify
    AddHandler swApp.BackgroundProcessingEndNotify, AddressOf Me.mySwApp_BackgroundProcessingEndNotify
End Sub

Private Function mySwApp_BackgroundProcessingStartNotify(ByVal filename As String) As Integer
    MsgBox("Background processing start event fired")
End Function

Private Function mySwApp_BackgroundProcessingEndNotify(ByVal filename As String) As Integer
    MsgBox("Background processing end event fired")
    swApp.EnableBackgroundProcessing = False
End Function

Public Sub CheckBox1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles CheckBox1.Click
    checkBoxClicked = True
End Sub

Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles Button1.Click
    Close()
End Sub
End Class

```

Fire Notification When Changing a Table in a Drawing Document Example (C#)

This example shows how to fire a notification when a table is changed in a drawing document.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Select a drawing view and click Insert > Tables > Bill of Materials.
// 4. Click OK in the Bill of Materials PropertyManager page.
// 5. Run this macro (press F5).
// 6. Right-click a column in the table and select Delete > Column.
//
// Postconditions:
// 1. Displays a message box informing you that the table was modified.
//    NOTE: Check the taskbar for the message box if you don't
//    see it.
// 2. Click OK to close the message box.
// 3. Stop the debugger.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace ModifyTableNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public DrawingDoc swDrawingDoc;
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swDrawingDoc = (DrawingDoc)swModel;
            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swDrawingDoc.ModifyTableNotify += this.swDrawingDoc_ModifyTableNotify;
        }

        private int swDrawingDoc_ModifyTableNotify(TableAnnotation TableAnnotation, int TableType, int reason, int RowInfo, int ColumnInfo, String
DataInfo)
        {
            MessageBox.Show("A table was modified. Title: " + TableAnnotation.Title + ", Type: " + TableType + ", Reason code: " + reason +
            ", RowInfo: " + RowInfo + ", ColumnInfo: " + ColumnInfo + ", DataInfo: " + DataInfo);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Changing a Table in a Drawing Document Example (VBA)

This example shows how to fire a notification when a table is changed in a drawing document:

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Copy Main to the main module.  
' 3. Click Insert > Class Module and copy Class to that module.  
' 4. Select a drawing view and click Insert > Tables > Bill of Materials.  
' 5. Click OK in the Bill of Materials PropertyManager page.  
' 6. Run this macro (press F5).  
' 7. Right-click a column and select Delete > Column.  
'  
' Postconditions:  
' 1. Displays a message box informing you that the table was modified.  
' 2. Click OK to close the message box.  
'-----
```

'Main

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDraw As SldWorks.DrawingDoc  
Dim errorstatus As Long, warningstatus As Long  
Dim swDrawEvents As Class1  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
  
'Set up events  
Set swDraw = swModel  
Set swDrawEvents = New Class1  
Set swDrawEvents.swDraw = swApp.ActiveDoc  
  
End Sub
```

[Back to top](#)

'Class

Option Explicit

```
Public WithEvents swDraw As SldWorks.DrawingDoc  
  
Private Function swDraw_ModifyTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As Long, ByVal reason  
As Long, ByVal RowInfo As Long, ByVal ColumnInfo As Long, ByVal DataInfo As String) As Long  
MsgBox "A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ", RowInfo:  
" & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo  
End Function
```

[Back to top](#)

Fire Notification When Changing a Table in a Drawing Document Example (VB.NET)

This example shows how to fire a notification when a table is changed in a drawing document.

```

'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Verify that the Tools > Options > System Options >
'   Stop VSTA debugger on macro exit check box is not selected.
' 3. Select a drawing view and click Insert > Tables > Bill of Materials.
' 4. Click OK in the Bill of Materials PropertyManager page.
' 5. Run this macro (press F5).
' 6. Right-click a column in the table and select Delete > Column.
'
' Postconditions:
' 1. Displays a message box informing you that the table was modified.
'   NOTE: Check the taskbar for the message box if you don't
'   see it.
' 2. Click OK to close the message box.
' 3. Stop the debugger.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swDrawingDoc As DrawingDoc

    Public Sub Main()

        Dim swModel As ModelDoc2

        swModel = swApp.ActiveDoc

        ' Set up event
        swDrawingDoc = swModel
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swDrawingDoc.ModifyTableNotify, AddressOf Me.swDrawingDoc_ModifyTableNotify
    End Sub

    Private Function swDrawingDoc_ModifyTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As Integer, ByVal
reason As Integer, ByVal RowInfo As Integer, ByVal ColumnInfo As Integer, ByVal DataInfo As String) As Integer
        MsgBox("A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ",
RowInfo: " & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo)
    End Function

    Public swApp As SldWorks
End Class

```

Fire Notification When Changing a Table in a Part Document Example (C#)

This example shows how to fire a notification when a table is changed in a part document.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Click Insert > Tables > Bill of Materials.
// 4. Click OK in the Bill of Materials PropertyManager page and
//    click OK again.
// 5. Run this macro (press F5).
// 6. Right-click a column in the table and select Delete > Column.
//
// Postconditions:
// 1. Displays a message box informing you that the table was modified.
//    NOTE: Check the taskbar for the message box if it was not displayed.
// 2. Click OK to close the message box.
// 3. Stop the debugger.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace ModifyTableNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPartDoc;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swPartDoc = (PartDoc)swModel;
            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swPartDoc.ModifyTableNotify += this.swPartDoc_ModifyTableNotify;
        }

        private int swPartDoc_ModifyTableNotify(TableAnnotation TableAnnotation, int TableType, int reason, int RowInfo, int
        ColumnInfo, String DataInfo)
        {
            MessageBox.Show("A table was modified. Title: " + TableAnnotation.Title + ", Type: " + TableType + ", Reason code: " +
            reason + ", RowInfo: " + RowInfo + ", ColumnInfo: " + ColumnInfo + ", DataInfo: " + DataInfo);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Changing a Table in a Part Document Example (VBA)

This example shows how to fire a notification when a table is changed in a part document.

```
'-----
' Preconditions:
' 1. Open a part document.
' 2. Copy Main to the main module.
' 3. Click Insert > Class Module and copy Class to that module.
' 4. Click Insert > Tables > Bill of Materials.
' 5. Click OK in the Bill of Materials PropertyManager page and
'    click OK again.
' 6. Run this macro (press F5).
' 7. Right-click a column in the table and select Delete > Column.
'
' Postconditions:
' 1. Displays a message box informing you that the table was modified.
' 2. Click OK to close the message box.
'-----
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim errorstatus As Long, warningstatus As Long
Dim swPartEvents As Class1
```

```
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    'Set up events
    Set swPart = swModel
    Set swPartEvents = New Class1
    Set swPartEvents.swPart = swApp.ActiveDoc
End Sub
```

[Back to top](#)

'Class

Option Explicit

```
Public WithEvents swPart As SldWorks.PartDoc
```

```
Private Function swPart_ModifyTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As Long, ByVal reason
As Long, ByVal RowInfo As Long, ByVal ColumnInfo As Long, ByVal DataInfo As String) As Long
MsgBox "A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ", RowInfo:
" & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo
End Function
```

[Back to top](#)

Fire Notification When Changing a Table in a Part Document Example (VB.NET)

This example shows how to fire a notification when a table is changed in a part document.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Verify that the Tools > Options > System Options >
'    Stop VSTA debugger on macro exit check box is not selected.
' 3. Click Insert > Tables > Bill of Materials.
' 4. Click OK in the Bill of Materials PropertyManager page and
'    click OK again.
' 5. Run this macro (press F5).
' 6. Right-click a column in the table and select Delete > Column.
'
' Postconditions:
' 1. Displays a message box informing you that the table was modified.
'    NOTE: Check the taskbar for the message box if it is not displayed.
' 2. Click OK to close the message box.
' 3. Stop the debugger.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swPartDoc As PartDoc
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Set up event
        swPartDoc = swModel
        AttachEventHandlers()
```

```
    End Sub
```

```
    Sub AttachEventHandlers()
```

```
        AttachSWEvents()
```

```
    End Sub
```

```
    Sub AttachSWEvents()
```

```
        AddHandler swPartDoc.ModifyTableNotify, AddressOf Me.swPartDoc_ModifyTableNotify
```

```
    End Sub
```

```
    Private Function swPartDoc_ModifyTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As Integer, ByVal reason
As Integer, ByVal RowInfo As Integer, ByVal ColumnInfo As Integer, ByVal DataInfo As String) As Integer
```

```
        MsgBox("A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ",
RowInfo: " & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo)
```

```
    End Function
```

```
    Public swApp As SldWorks
```

```
End Class
```

Fire Notification When Changing a Table in an Assembly Document Example (C#)

This example shows how to fire a notification when a table is changed in an assembly document.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Click Insert > Tables > Bill of Materials.
// 4. Click OK in the Bill of Materials PropertyManager page and
//    click OK again.
// 5. Run this macro (press F5).
// 6. Right-click on a column in the table and select Delete > Column.
//
// Postconditions:
// 1. Displays a message box informing you that the table was modified.
//    NOTE: Check the taskbar for the message box if you don't
//    see it.
// 2. Click OK to close the message box.
// 3. Stop the debugger..
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace ModifyTableNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public AssemblyDoc swAssemblyDoc;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Hashtable openAssem = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swAssemblyDoc = (AssemblyDoc)swModel;
            openAssem = new Hashtable();

            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssemblyDoc.ModifyTableNotify += this.swAssemblyDoc_ModifyTableNotify;
        }

        private int swAssemblyDoc_ModifyTableNotify(TableAnnotation TableAnnotation, int TableType, int reason, int RowInfo, int
        ColumnInfo, String DataInfo)
        {
            MessageBox.Show("A table was modified. Title: " + TableAnnotation.Title + ", Type: " + TableType + ", Reason code: " +
            reason + ", RowInfo: " + RowInfo + ", ColumnInfo: " + ColumnInfo + ", DataInfo: " + DataInfo);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Changing a Table in an Assembly Document Example (VBA)

This example shows how to fire a notification when a table is changed in an assembly document.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Copy Main to the main module.  
' 3. Click Insert > Class Module and copy Class to that module.  
' 4. Click Insert > Tables > Bill of Materials.  
' 5. Click OK in the Bill of Materials PropertyManager page  
'    click OK again.  
' 6. Run this macro (press F5).  
' 7. Right-click a column in the table and select Delete > Column.  
,  
' Postconditions:  
' 1. Displays a message box informing you that the table was modified.  
' 2. Click OK to close the message box.  
'-----  
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssem As SldWorks.AssemblyDoc  
Dim errorstatus As Long, warningstatus As Long  
Dim swAssemEvents As Class1
```

```
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    'Set up events  
    Set swAssem = swModel  
    Set swAssemEvents = New Class1  
    Set swAssemEvents.swAssem = swApp.ActiveDoc  
End Sub
```

[Back to top](#)

'Class

Option Explicit

```
Public WithEvents swAssem As SldWorks.AssemblyDoc
```

```
Private Function swAssem_ModifyTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As Long, ByVal  
reason As Long, ByVal RowInfo As Long, ByVal ColumnInfo As Long, ByVal DataInfo As String) As Long  
MsgBox "A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ", RowInfo:  
" & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo  
End Function
```

[Back to top](#)

Fire Notification When Changing a Table in an Assembly Document Example (VB.NET)

This example shows how to fire a notification when a table is changed in an assembly document.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Verify that the Tools > Options > System Options >
'    Stop VSTA debugger on macro exit check box is not selected.
' 3. click Insert > Tables > Bill of Materials.
' 4. Click OK in the Bill of Materials PropertyManager page and
'    click OK again.
' 5. Run this macro (press F5).
' 6. Right-click a column in the table and select Delete > Column.
'
' Postconditions:
' 1. Displays a message box informing you that the table was modified.
'    NOTE: Check the taskbar for the message box if it is not displayed.
' 2. Click OK to close the message box.
' 3. Stop the debugger.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swAssemblyDoc As AssemblyDoc
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
        Dim openAssem As Hashtable

        swModel = swApp.ActiveDoc

        ' Set up event
        swAssemblyDoc = swModel
        openAssem = New Hashtable
        AttachEventHandlers()
```

```
    End Sub
```

```
    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub
```

```
    Sub AttachSWEvents()
        AddHandler swAssemblyDoc.ModifyTableNotify, AddressOf Me.swAssemblyDoc_ModifyTableNotify
    End Sub
```

```
    Private Function swAssemblyDoc_ModifyTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As Integer, ByVal
reason As Integer, ByVal RowInfo As Integer, ByVal ColumnInfo As Integer, ByVal DataInfo As String) As Integer
        MsgBox("A table was modified. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", Reason code: " & reason & ",
RowInfo: " & RowInfo & ", ColumnInfo: " & ColumnInfo & ", DataInfo: " & DataInfo)
    End Function
```

```
    Public swApp As SldWorks
```

```
End Class
```

Fire Notification When Changing Configuration of Reference Component Example (VB.NET)

This example shows how to fire an event when changing the configuration of a reference component in an assembly.

```

'-----
' Preconditions:
' 1. Open an assembly document that contains at least one
'   subassembly (i.e., reference component) that has multiple
'   configurations.
' 2. Make sure that the Tools > Options > Stop VSTA debugger on macro exit
'   checkbox is not selected.
' 3. Run this macro (press F5).
' 4. Right-click a subassembly and select Configure Component.
' 5. In the Configuration column on the Modify Configurations
'   dialog, select a different configuration and click OK.
'
' Postconditions:
' 1. Displays a message box informing you that the component's
'   configuration is being changed. Check the taskbar for
'   the message box.
' 2. Click OK to close the message box and
'   the Modify Configurations dialog.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swAssemblyDoc As AssemblyDoc

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim openAssem As Hashtable

        swModel = swApp.ActiveDoc

        ' Set up event
        swAssemblyDoc = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssemblyDoc.ComponentConfigurationChangeNotify, AddressOf Me.swAssemblyDoc_ComponentConfigurationChangeNotify
    End Sub

    Private Function swAssemblyDoc_ComponentConfigurationChangeNotify(ByVal componentName As String, ByVal oldConfigurationName As String, ByVal newConfigurationName As String) As Integer
        ' Display message when user is changing a reference component's configuration
        MsgBox("A component's configuration is being changed. Component name: " & componentName & ", old configuration name: " & oldConfigurationName & ", and new configuration name: " & newConfigurationName)
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Fire Notification When Changing Configuration of Reference Component Example (C#)

This example shows how to fire an event when changing the configuration of a reference component in an assembly.

```
//-----
// Preconditions:
// 1. Open an assembly document that contains at least one
// subassembly (i.e., reference component) that has
// multiple configurations.
// 2. Make sure that the Tools > Options > Stop VSTA debugger on macro exit
// checkbox is not selected.
// 3. Run this macro (press F5).
// 4. Right-click a subassembly and select Configure Component.
// 5. In the Configuration column on the Modify Configurations
// dialog, select a different configuration and click OK.
//
// Postconditions:
// 1. Displays a message box informing you that the component's
// configuration is being changed. Check the taskbar for the
// message box.
// 2. Click OK to close the message box and
// the Modify Configurations dialog.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Windows.Forms;
using System.Collections;

namespace CompConfigChangeNotifyAssemblyDocCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssem;
        public void Main()
        {
            ModelDoc2 swModel;
            Hashtable openAssem;
            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Set up events
            swAssem = (AssemblyDoc)swModel;
            openAssem = new Hashtable();
            AttachEventHandlers();
        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssem.ComponentConfigurationChangeNotify += this.swAssem_ComponentConfigurationChangeNotify;
        }

        public int swAssem_ComponentConfigurationChangeNotify(string componentName, string oldConfigurationName, string newConfigurationName)
        //Send message when user is changing the configuration of a reference component
        {
            MessageBox.Show("A component's configuration is being changed." + " Component name: " + componentName + ", old configuration name: " + oldConfigurationName + ", and new configuration name: " + newConfigurationName);
            return 1;
        }
    }
}
```

Fire Notification When Changing Configuration of Reference Component Example (VBA)

This example shows how to fire an event when changing the configuration of a reference component in an assembly.

```

'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class Module and copy and past this code
'    the class module.
' 3. Open an assembly document that contains at least one
'    subassembly (i.e., reference component) that has
'    multiple configurations.
' 4. Run this macro (press F5).
' 5. Right-click a subassembly and select Configure Component.
' 6. In the Configuration column on the Modify Configurations
'    dialog, select a different configuration and click OK.
'
' Postconditions:
' 1. Displays a message box informing you that a component's
'    configuration is being changed.
' 2. Click OK to close the message box and
'    the Modify Configurations dialog.
'-----
'Main module

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssem As SldWorks.AssemblyDoc
Dim swAssemEvents As Class1

Sub main()

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc

'Set up events
Set swAssem = swModel
Set swAssemEvents = New Class1
Set swAssemEvents.swAssem = swApp.ActiveDoc

End Sub

'Class module

Option Explicit

Public WithEvents swAssem As SldWorks.AssemblyDoc

'Send message when user is changing the configuration of a reference component
Private Function swAssem_ComponentConfigurationChangeNotify(ByVal componentName As String, ByVal oldConfigurationName As String,
ByVal newConfigurationName As String) As Long

MsgBox "A component's configuration is being changed: Component name: " & componentName & ", previous configuration name: " &
oldConfigurationName & ", and new configuration name: " & newConfigurationName

End Function

```

Fire Notification When Component Referenced Display State Changes Example (VBA)

This example shows how to fire a notification when a component's referenced display state is changed in an assembly.

```
'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class Module and copy and paste this code
'    in the class module.
' 3. Open an assembly document that has
'    a component with multiple referenced
'    display states.
' 4. Run this macro (press F5)
' 5. Change the referenced display state of
'    a component (right-click the component, click the
'    Component Properties button, select a different
'    referenced display state, and click OK).
'
' Postconditions:
' 1. Displays a message box informing you that the referenced
'    display state of a component has changed.
' 2. Click OK to close the message box.
'-----
'Main module
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssem As SldWorks.AssemblyDoc
Dim swAssemEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Set up events
    Set swAssem = swModel
    Set swAssemEvents = New Class1
    Set swAssemEvents.swAssem = swApp.ActiveDoc

End Sub
```

'Class module

Option Explicit

Public WithEvents swAssem As SldWorks.AssemblyDoc

'Send message after user changes referenced display state of a component in an assembly

```
Private Function swAssem_ComponentReferredDisplayStateChangeNotify(ByVal componentModel As Object, ByVal compName As String, ByVal
oldDSId As Long, ByVal oldDSName As String, ByVal newDSID As Long, ByVal newDSName As String) As Long
    MsgBox "The component's referenced display state has changed."
End Function
```

Fire Notification When Component Referenced Display State Changes Example (C#)

This example shows how to fire a notification when a component's referenced display state is changed in an assembly.

```
//-----
// Preconditions:
// 1. Open an assembly document that has
//    a component with multiple referenced
//    display states.
// 2. Make sure that Tools > Options > Stop VSTA
//    debugger on macro exit is not selected.
// 3. Run this macro (press F5).
// 4. Change the referenced display state of
//    a component (right-click the component, click the
//    Component Properties button, select a different
//    referenced display state, and click OK).
//
// Postconditions:
// 1. Displays a message box informing you that the referenced
//    display state of a component has changed. Check the
//    taskbar for the message box.
// 2. Click OK to close the message box.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Windows.Forms;

namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssem;
        public void Main()
        {
            ModelDoc2 swModel;
            Hashtable openAssem;
            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Set up events
            swAssem = (AssemblyDoc)swModel;
            openAssem = new Hashtable();
            AttachEventHandlers();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssem.ComponentReferredDisplayStateChangeNotify += this.swAssem_ComponentReferredDisplayStateChangeNotify;
        }

        public int swAssem_ComponentReferredDisplayStateChangeNotify(object componentModel, string compName, int oldDSId, string oldDSName, int newDSId, string newDSName)
        {
            //Send message when user changes referenced display state of a component
            {
                MessageBox.Show("A component's referenced display state has changed.");
                return 1;
            }
        }
    }
}
```

Fire Notification When Component Referenced Display State Changes Example (VB.NET)

This example shows how to fire a notification when a component's referenced display state is changed in assembly.

```

-----
Preconditions:
1. Open an assembly document that has
   a component with multiple referenced
   display states.
2. Make sure that Tools > Options > Stop VSTA
   debugger on macro exit is not selected.
3. Run this macro (press F5)
4. Change the referenced display state of
   a component (right-click the component, click the
   Component Properties button, select a different
   referenced display state, and click OK).
-----
Postconditions:
1. Displays a message box informing you that the referenced
   display state of a component has changed. Check the
   taskbar for the message box.
2. Click OK to close the message box.
-----
Option Explicit On

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swAssem As AssemblyDoc

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim openAssem As Hashtable
        swModel = swApp.ActiveDoc

        'Set up events
        swAssem = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssem.ComponentReferredDisplayStateChangeNotify, AddressOf Me.swAssem_ComponentReferredDisplayStateChangeNotify
    End Sub

    Private Function swAssem_ComponentReferredDisplayStateChangeNotify(ByVal componentModel As Object, ByVal compName As String, ByVal oldSid As Integer,
        ByVal oldDSName As String, ByVal newDSId As Integer, ByVal newDSName As String) As Integer
        'Send message after user changes referenced display state of a component
        MsgBox("The referenced display state of a component has changed.")
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Fire Notification When Inserting a Table in a Drawing Document Example (C#)

This example shows how to fire a notification when a table is inserted in a drawing document.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Run this macro (press F5).
// 4. Select a drawing view.
// 5. Click Insert > Tables > Bill of Materials.
// 6. Click OK in the Bill of Materials PropertyManager page.
//
// Postconditions:
// 1. Displays a message box informing you that a table will be inserted.
// 2. Click OK to close the message box.
//    NOTE: Check the taskbar for the message box if you don't
//    see it.
// 3. Click to place the table.
// 4. Stop the debugger.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace TableInsertNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public DrawingDoc swDrawingDoc;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swDrawingDoc = (DrawingDoc)swModel;

            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swDrawingDoc.InsertTableNotify += this.swDrawingDoc_InsertTableNotify;
        }

        private int swDrawingDoc_InsertTableNotify(TableAnnotation TableAnnotation, string TableType, string TemplatePath)
        {
            path: " + TemplatePath);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Inserting a Table in a Drawing Document Example (VBA)

This example shows how to fire a notification when a table is inserted in a drawing document.

```
'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Copy Main to the main module.
' 3. Click Insert > Class Module and copy Class to that module.
' 4. Run this macro (press F5).
' 5. Select a drawing view.
' 6. Click Insert > Tables > Bill of Materials.
' 7. Click OK in the Bill of Materials PropertyManager page.
'
' Postconditions:
' 1. Displays a message box informing you that a table will be inserted.
' 2. Click OK to close the message box.
' 3. Click to place the table.
'-----
```

'Main

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim errorstatus As Long, warningstatus As Long
Dim swDrawEvents As Class1
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Set up events
    Set swDraw = swModel
    Set swDrawEvents = New Class1
    Set swDrawEvents.swDraw = swApp.ActiveDoc
```

End Sub

[Back to top](#)

'Class

Option Explicit

Public WithEvents swDraw As SldWorks.DrawingDoc

```
Private Function swDraw_InsertTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As String, ByVal TemplatePath As String) As Long
    MsgBox "A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " & TemplatePath
End Function
```

[Back to top](#)

Fire Notification When Inserting a Table in a Drawing Document Example (VB.NET)

This example shows how to fire a notification when a table is inserted in a drawing document.

```

'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Verify that the Tools > Options > System Options >
'   Stop VSTA debugger on macro exit check box is not selected.
' 3. Run this macro (press F5).
' 4. Select a drawing view.
' 5. Click Insert > Tables > Bill of Materials.
' 6. Click OK in the Bill of Materials PropertyManager page.
'
' Postconditions:
' 1. Displays a message box informing you that a table will be inserted.
' 2. Click OK to close the message box.
'   NOTE: Check the taskbar for the message box if you don't
'   see it.
' 3. Click to place the table.
' 4. Stop the debugger.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swDrawingDoc As DrawingDoc
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
```

```
        'Dim openDraw As Hashtable
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Set up event
```

```
        swDrawingDoc = swModel
```

```
        'openDraw = New Hashtable
```

```
        AttachEventHandlers()
```

```
    End Sub
```

```
    Sub AttachEventHandlers()
```

```
        AttachSwEvents()
```

```
    End Sub
```

```
    Sub AttachSwEvents()
```

```
        AddHandler swDrawingDoc.InsertTableNotify, AddressOf Me.swDrawingDoc_InsertTableNotify
```

```
    End Sub
```

```
    Private Function swDrawingDoc_InsertTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As String, ByVal
TemplatePath As String) As Integer
```

```
        MsgBox("A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " &
TemplatePath)
```

```
    End Function
```

```
    Public swApp As SldWorks
```

```
End Class
```

Fire Notification When Inserting a Table in a Part Document Example (C#)

This example shows how to fire a notification when a table is inserted in a part document.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Run this macro (press F5).
// 4. Click Insert > Tables > Bill of Materials.
// 5. Click OK in the Bill of Materials PropertyManager page and
//    click OK again.
//
// Postconditions:
// 1. Displays a message box informing you that a table
//    will be inserted in the part.
// 2. Click OK to close the message box.
//    NOTE: Check the taskbar for the message box if you don't
//    see it.
// 3. Click to place the table.
// 4. Stop the debugger.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace TableInsertNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPartDoc;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swPartDoc = (PartDoc)swModel;
            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swPartDoc.InsertTableNotify += this.swPartDoc_InsertTableNotify;
        }

        private int swPartDoc_InsertTableNotify(TableAnnotation TableAnnotation, int TableType, string TemplatePath)
        {
            MessageBox.Show("A table will be inserted. Title: " + TableAnnotation.Title + ", Type: " + TableType + ", and Template
path: " + TemplatePath);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Inserting a Table in a Part Document Example (VBA)

This example shows how to fire a notification when a table is inserted in a part document.

```
'-----  
' Preconditions:  
' 1. Open a part document.  
' 2. Copy Main to the main module.  
' 3. Click Insert > Class Module and copy Class to that module.  
' 4. Run this macro (press F5).  
' 5. Click Insert > Tables > Bill of Materials.  
' 6. Click OK in the Bill of Materials PropertyManager page and  
'    click OK again.  
'  
' Postconditions:  
' 1. Displays a message box informing you that a table  
'    will be inserted in the part.  
' 2. Click OK to close the message box.  
' 3. Click to place the table.  
'-----  
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swPart As SldWorks.PartDoc  
Dim errorstatus As Long, warningstatus As Long  
Dim swPartEvents As Class1
```

```
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Set up events  
    Set swPart = swModel  
    Set swPartEvents = New Class1  
    Set swPartEvents.swPart = swApp.ActiveDoc  
  
End Sub
```

[Back to top](#)

Class

Option Explicit

```
Public WithEvents swPart As SldWorks.PartDoc
```

```
Private Function swPart_InsertTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As Long, ByVal  
TemplatePath As String) As Long  
MsgBox "A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " &  
TemplatePath  
End Function
```

[Back to top](#)

Fire Notification When Inserting a Table in a Part Document Example (VB.NET)

This example shows how to fire a notification when a table is inserted in a part document.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Verify that the Tools > Options > System Options >
'    Stop VSTA debugger on macro exit check box is not selected.
' 3. Run this macro (press F5).
' 4. Click Insert > Tables > Bill of Materials.
' 5. Click OK in the Bill of Materials PropertyManager page and
'    click OK again.
'
' Postconditions:
' 1. Displays a message box informing you that a table
'    will be inserted in the part.
' 2. Click OK to close the message box.
'    NOTE: Check the taskbar for the message box if you don't
'    see it.
' 3. Click to place the table.
' 4. Stop the debugger.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swPartDoc As PartDoc

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim openAssem As Hashtable

        swModel = swApp.ActiveDoc

        ' Set up event
        swPartDoc = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swPartDoc.InsertTableNotify, AddressOf Me.swPartDoc_InsertTableNotify
    End Sub

    Private Function swPartDoc_InsertTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As Integer, ByVal
TemplatePath As String) As Integer
        MsgBox("A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " &
TemplatePath)
    End Function

    Public swApp As SldWorks
End Class

```

Fire Notification When Inserting a Table in an Assembly Document Example (C#)

This example shows how to fire a notification when a table is inserted in an assembly document.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Verify that the Tools > Options > System Options >
//    Stop VSTA debugger on macro exit check box is not selected.
// 3. Run this macro (press F5).
// 4. Click Insert > Tables > Bill of Materials.
// 5. Click OK in the Bill of Materials PropertyManager page and
//    click OK again.
//
// Postconditions:
// 1. Displays a message box informing you that a table will be
//    inserted in the assembly.
//    NOTE: Check the taskbar for the message box if you don't
//    see it.
// 2. Click OK to close the message box.
// 3. Click to place the table.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
using System.Windows.Forms;
namespace TableInsertNotification_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public AssemblyDoc swAssemblyDoc;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Hashtable openAssem = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event
            swAssemblyDoc = (AssemblyDoc)swModel;
            openAssem = new Hashtable();

            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssemblyDoc.InsertTableNotify += this.swAssemblyDoc_InsertTableNotify;
        }

        private int swAssemblyDoc_InsertTableNotify(TableAnnotation TableAnnotation, string TableType, string TemplatePath)
        {
            path: " + TemplatePath);
            return 0;
        }

        public SldWorks swApp;
    }
}
```

Fire Notification When Inserting a Table in an Assembly Document Example (VBA)

This example shows how to fire a notification when a table is inserted in an assembly document.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Copy Main to the main module.  
' 3. Click Insert > Class Module and copy Class to that module.  
' 4. Run this macro (press F5).  
' 5. Click Insert > Tables > Bill of Materials.  
' 6. Click OK in the Bill of Materials PropertyManager page and  
'    click OK again.  
'  
' Postconditions:  
' 1. Displays a message box informing you that a table will be  
'    inserted in the assembly.  
' 2. Click OK to close the message box.  
' 3. Click to place the table.  
'-----  
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssem As SldWorks.AssemblyDoc  
Dim errorstatus As Long, warningstatus As Long  
Dim swAssemEvents As Class1
```

Sub main()

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc
```

```
'Set up events  
Set swAssem = swModel  
Set swAssemEvents = New Class1  
Set swAssemEvents.swAssem = swApp.ActiveDoc
```

End Sub

[Back to top](#)

[Class](#)

Option Explicit

```
Public WithEvents swAssem As SldWorks.AssemblyDoc
```

```
Private Function swAssem_InsertTableNotify(ByVal TableAnnotation As SldWorks.ITableAnnotation, ByVal TableType As String, ByVal  
TemplatePath As String) As Long  
    MsgBox "A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " &  
TemplatePath  
End Function
```

[Back to top](#)

Fire Notification When Inserting a Table in an Assembly Document Example (VB.NET)

This example shows how to fire a notification when a table is inserted in an assembly document.

```

'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Verify that the Tools > Options > System Options >
'   Stop VSTA debugger on macro exit check box is not selected.
' 3. Run this macro (press F5).
' 4. Click Insert > Tables > Bill of Materials.
' 5. Click OK in the Bill of Materials PropertyManager page and
'   click OK again.
'
' Postconditions:
' 1. Displays a message box informing you that a table will be
'   inserted in the assembly.
'   NOTE: Check the taskbar for the message box if you don't
'   see it.
' 2. Click OK to close the message box.
' 3. Click to place the table.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swAssemblyDoc As AssemblyDoc
```

```
    Public Sub Main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim openAssem As Hashtable
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Set up event
```

```
        swAssemblyDoc = swModel
```

```
        openAssem = New Hashtable
```

```
        AttachEventHandlers()
```

```
    End Sub
```

```
    Sub AttachEventHandlers()
```

```
        AttachSwEvents()
```

```
    End Sub
```

```
    Sub AttachSwEvents()
```

```
        AddHandler swAssemblyDoc.InsertTableNotify, AddressOf Me.swAssemblyDoc_InsertTableNotify
```

```
    End Sub
```

```
    Private Function swAssemblyDoc_InsertTableNotify(ByVal TableAnnotation As TableAnnotation, ByVal TableType As String, ByVal
TemplatePath As String) As Integer
```

```
        MsgBox("A table will be inserted. Title: " & TableAnnotation.Title & ", Type: " & TableType & ", and Template path: " &
TemplatePath)
```

```
    End Function
```

```
    Public swApp As SldWorks
```

```
End Class
```

Fire Notification When Publishing Part to MBD 3D PDF Example (C#)

This example shows how to fire a notification when publishing a part document to SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Verify that:
//   * specified part,
//   * SOLIDWORKS MBD 3D PDF theme, and
//   * c:\temp exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Gets the MBD3DPdfData object.
// 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
// 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
// 5. Sets standard views for the SOLIDWORKS MBD 3D PDF.
// 6. Publishes the part document to SOLIDWORKS MBD 3D PDF.
// 7. Displays a message saying that the part document
//   was published to SOLIDWORKS MBD 3D PDF.
// 8. Click OK to close the message box.
// 9. Examine the Immediate window and c:\temp\MBDPart1.PDF.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Diagnostics;
using System.Windows.Forms;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPart;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);
            string fileName = null;
            object standardViews = null;
            int[] viewIDs = new int[3];
            int status = 0;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Set up events
            swPart = (PartDoc)swModel;
            AttachEventHandlers();

            //Get MBD3DPdfData object
            swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

            //Set path and file name for SOLIDWORKS MBD 3D PDF
            swMBDPdfData.FilePath = "c:\\temp\\MBDPart1.PDF";

            //Set SOLIDWORKS MBD 3D PDF theme
            swMBDPdfData.ThemeName = "C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\data\\themes\\simple part (a4, portrait)\\theme.xml";

            //Set standard views for SOLIDWORKS MBD 3D PDF
            viewIDs[0] = (int)swStandardViews_e.swFrontView;
            viewIDs[1] = (int)swStandardViews_e.swTopView;
            viewIDs[2] = (int)swStandardViews_e.swDimetricView;
            standardViews = (object)viewIDs;
            swMBDPdfData.SetStandardViews(standardViews);

            //Publish part document to SOLIDWORKS MBD 3D PDF
            status = swModelDocExt.PublishTo3DPDF(swMBDPdfData);
            Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " + status);
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swPart.PublishTo3DPDFNotify += this.swPart_PublishTo3DPDFNotify;
        }

        private int swPart_PublishTo3DPDFNotify(string path)
        {
            MessageBox.Show("Part document published to SOLIDWORKS MBD 3D PDF: " + path);
            return 1;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Fire Notification When Publishing Part to MBD 3D PDF Example (VBA)

This example shows how to fire a notification when publishing a part document to SOLIDWORKS MBD 3D PDF.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified part,
'   * SOLIDWORKS MBD 3D PDF theme, and
'   * c:\temp exist.
' 2. Copy this code to the main module.
' 3. Click Insert > Class Module and copy this code to the
'   Class1 module.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Gets the MBD3DPdfData object.
' 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 5. Sets standard views for the SOLIDWORKS MBD 3D PDF.
' 6. Publishes the part document to SOLIDWORKS MBD 3D PDF.
' 7. Displays a message saying that the part document
'   was published to SOLIDWORKS MBD 3D PDF.
' 8. Click OK to close the message box.
' 9. Examine the Immediate window and c:\temp\MBDPart1.PDF.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
'main module
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As ModelDocExtension
Dim swMBDPdfData As SldWorks.MBD3DPdfData
Dim fileName As String
Dim standardViews As Variant
Dim viewIDs(2) As Long
Dim status As Long
Dim errors As Long
Dim warnings As Long
Dim swPart As SldWorks.PartDoc
Dim swPartEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Event notification
    Set swPart = swModel
    Set swPartEvents = New Class1
    Set swPartEvents.swPart = swApp.ActiveDoc

    'Get MBD3DPdfData object
    Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData

    'Specify path and file name for SOLIDWORKS MBD 3D PDF
    swMBDPdfData.filePath = "c:\temp\MBDPart1.PDF"

    'Set SOLIDWORKS MBD 3D PDF theme
    swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\theme.xml"

    'Set standard views for SOLIDWORKS MBD 3D PDF
    viewIDs(0) = swStandardViews_e.swFrontView
    viewIDs(1) = swStandardViews_e.swTopView
    viewIDs(2) = swStandardViews_e.swDimetricView
    standardViews = viewIDs
    swMBDPdfData.SetStandardViews (standardViews)

    'Publish part document to SOLIDWORKS MBD 3D PDF
    status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
    Debug.Print ("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

End Sub

'Class1 module
Option Explicit

Public WithEvents swPart As SldWorks.PartDoc

Private Function swPart_PublishTo3DPDFNotify(ByVal path As String) As Long
    MsgBox "Part document published to SOLIDWORKS MBD 3D PDF: " & path
End Function

```

[Back to top](#)

[Back to top](#)

Fire Notification When Publishing Part to MBD 3D PDF Example (VB.NET)

This example shows how to fire a notification when publishing a part document to SOLIDWORKS MBD 3D PDF.

```

'-----
' Preconditions:
' 1. Verify that:
'    * specified part,
'    * SOLIDWORKS MBD 3D PDF theme, and
'    * c:\temp exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Gets the MBD3DPdfData object.
' 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 4. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 5. Sets standard views for the SOLIDWORKS MBD 3D PDF.
' 6. Publishes the part document to SOLIDWORKS MBD 3D PDF.
' 7. Displays a message saying that the part document
'    was published to SOLIDWORKS MBD 3D PDF.
' 8. Click OK to close the message box.
' 9. Examine the Immediate window and c:\temp\MBDPart1.PDF.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public WithEvents swPart As PartDoc

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMBDPdfData As MBD3DPdfData
        Dim fileName As String
        Dim standardViews As Object
        Dim viewIDs(2) As Integer
        Dim status As Integer
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Set up events
        swPart = swModel
        AttachEventHandlers()

        'Get MBD3DPdfData object
        swMBDPdfData = swModelDocExt.GetMBD3DPdfData

        'Set path and file name for SOLIDWORKS MBD 3D PDF
        swMBDPdfData.FilePath = "c:\temp\MBDPart1.PDF"

        'Set SOLIDWORKS MBD 3D PDF theme
        swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\theme.xml"

        'Set standard views for SOLIDWORKS MBD 3D PDF
        viewIDs(0) = swStandardViews_e.swFrontView
        viewIDs(1) = swStandardViews_e.swTopView
        viewIDs(2) = swStandardViews_e.swDimetricView
        standardViews = viewIDs
        swMBDPdfData.SetStandardViews(standardViews)

        'Publish part document to SOLIDWORKS MBD 3D PDF
        status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
        Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swPart.PublishTo3DPDFNotify, AddressOf Me.swPart_PublishTo3DPDFNotify
    End Sub

    Private Function swPart_PublishTo3DPDFNotify(ByVal path As String) As Integer
        MsgBox("Part document published to SOLIDWORKS MBD 3D PDF: " & path)
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Fire Notification When Activating a Sheet Example (C#)

This example shows how to fire a notification when a sheet is activated in a drawing document.

```
//-----
//Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
//
// Postconditions:
// 1. Displays a message box informing you that a sheet is about to be
//    activated.
// 2. Click OK to close the message box.
//
// NOTES:
// * The IDrawingDoc ActivateSheetPostNotify event only fires
//   when a sheet is interactively activated in SOLIDWORKS; this event
//   does not fire when a sheet is activated through the API.
// * Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace ActivateSheetEventsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public DrawingDoc swDrawingDoc;

        public void Main()
        {
            ModelDoc2 swModel;
            Boolean status;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Set up events
            swDrawingDoc = (DrawingDoc)swModel;
            AttachEventHandlers();

            // Activate Sheet4
            status = swDrawingDoc.ActivateSheet("Sheet4");
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swDrawingDoc.ActivateSheetPreNotify += this.swDraw_ActivateSheetPreNotify;
        }

        private int swDraw_ActivateSheetPreNotify(string SheetName)
        {
            MessageBox.Show("A sheet is about to be activated.");
            return 1;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Fire Notification When Activating a Sheet Example (VBA)

This example shows how to fire a notification when a sheet is activated in a drawing document.

```
'-----  
' Preconditions:  
' 1. Copy Main into the macro.  
' 2. Click Insert > Class Module and copy Class1 into that  
'    module.  
' 3. Open a drawing with two or more sheets.  
'-----  
' Postconditions:  
' 1. Click an inactive sheet's tab to activate it.  
' 2. Displays a message box informing you that a sheet is about  
'    to be activated.  
' 3. Click OK to close the message box.  
' 4. Displays a message box informing you that a sheet was activated.  
' 5. Click OK to close the message box.  
'-----  
  
'Main  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDraw As SldWorks.DrawingDoc  
Dim swDrawEvents As Class1  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Set up events  
    Set swDraw = swModel  
    Set swDrawEvents = New Class1  
    Set swDrawEvents.swDraw = swApp.ActiveDoc  
  
End Sub
```

[Back to top](#)

```
'Class1  
  
Option Explicit  
  
Public WithEvents swDraw As SldWorks.DrawingDoc  
  
Private Function swDraw_ActivateSheetPreNotify(ByVal SheetName As String) As Long  
    MsgBox "A sheet is about to be activated."  
End Function  
  
Private Function swDraw_ActivateSheetPostNotify(ByVal SheetName As String) As Long  
    MsgBox "A sheet was activated."  
End Function
```

[Back to top](#)

Fire Notification When Activating a Sheet Example (VB.NET)

This example shows how to fire a notification when a sheet is activated in a drawing document.

```

-----
Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
Postconditions:
1. Displays a message box informing you that a sheet is about to be
   activated.
2. Click OK to close the message box.
NOTES:
* The IDrawingDoc ActivateSheetPostNotify event only fires
  when a sheet is interactively activated in SOLIDWORKS; this event
  does not fire when a sheet is activated through the API.
* Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Windows.Forms
Imports System.Collections
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public WithEvents swDrawingDoc As DrawingDoc

    Dim swModel As ModelDoc2
    Dim status As Boolean

    Sub Main()

        swModel = swApp.ActiveDoc

        'Set up events
        swDrawingDoc = swModel
        AttachEventHandlers()

        ' Activate Sheet4
        status = swDrawingDoc.ActivateSheet("Sheet4")

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swDrawingDoc.ActivateSheetPreNotify, AddressOf Me.swDraw_ActivateSheetPreNotify
    End Sub

    Function swDraw_ActivateSheetPreNotify(ByVal SheetName As String) As Integer
        MsgBox("A sheet is about to be activated.")
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Fire Notifications When Renaming Components Example (C#)

This example shows how to fire notifications when you:

- are about to rename a component.
- rename a component.

```
//-----
// Preconditions:
// 1. Verify that these documents exist in public_documents\samples\tutorial\api:
//    * beam_boltconnection.sldasm
//    * beam with holes.sldprt
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Open public_documents\samples\tutorial\api\beam_boltconnection.sldasm.
// 2. Fires pre-notification before appending
//    123 to each assembly component's name.
// 3. Fires notification when appending 123 to
//    each assembly component's name.
// 4. Examine the FeatureManager design tree and the
//    Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do
// not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssy;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConfig = default(Configuration);
            Component2 swRootComp = default(Component2);
            object[] Children = null;
            Component2 swChild = default(Component2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            int ChildCount = 0;
            string oldName = null;
            string newName = null;
            bool bOldSetting = false;
            bool bRet = false;
            int i = 0;
            Hashtable openAssem = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;

            // Set up event
            swAssy = (AssemblyDoc)swModel;
            openAssem = new Hashtable();
            AttachEventHandlers();

            swConfigMgr = swModel.ConfigurationManager;
            swConfig = swConfigMgr.ActiveConfiguration;
            swRootComp = swConfig.GetRootComponent3(true);
            bOldSetting = swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swExtRefUpdateCompNames);
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swExtRefUpdateCompNames, false);
            Children = (object[])swRootComp.GetChildren();
            ChildCount = Children.Length;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            for (i = 0; i <= ChildCount - 1; i++)
            {
                swChild = (Component2)Children[i];
                // Changing component name requires component to be selected
                bRet = swChild.Select4(false, swSelData, false);
                oldName = swChild.Name2;
                newName = oldName + " 123";
                swChild.Name2 = newName;
            }
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swExtRefUpdateCompNames, bOldSetting);
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssy.PreRenameItemNotify += this.swAssy_PreRenameItemNotify;
            swAssy.RenameItemNotify += this.swAssy_RenameItemNotify;
        }

        private int swAssy_PreRenameItemNotify(int EntityType, string oldName, string newName)
        {
            Debug.Print("PRE-NOTIFICATION - about to rename component: " + oldName);
            return 0;
        }

        private int swAssy_RenameItemNotify(int EntityType, string oldName, string newName)
        {
            Debug.Print("NOTIFICATION - rename component: " + newName);
            return 0;
        }
    }
}
```

```
/// <summary>
///   The Sldworks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Fire Notifications When Renaming Components Example (VBA)

This example shows how to fire notifications when you:

- are about to rename a component.
- rename a component.

```

'-----
' Preconditions:
' 1. Copy and paste Main in the main module.
' 2. Click Insert > Class module and copy and paste Class1 in the class
'    module.
' 3. Verify that these documents exist in public_documents\samples\tutorial\api:
'    * beam_boltconnection.sldasm
'    * beam with holes.sldprt
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Open public_documents\samples\tutorial\api\beam_boltconnection.sldasm.
' 2. Fires pre-notification before appending
'    123 to each assembly component's name.
' 3. Fires notification when appending 123 to
'    each assembly component's name.
' 4. Examine the FeatureManager design tree and the
'    Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do
' not save changes.
'-----
'Main
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim Children As Variant
    Dim swChild As SldWorks.Component2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swAssyEvents As Class1
    Dim ChildCount As Long
    Dim oldName As String
    Dim newName As String
    Dim bOldSetting As Boolean
    Dim bRet As Boolean
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel

    'Set up events
    Set swAssy = swModel
    Set swAssyEvents = New Class1
    Set swAssyEvents.swAssy = swApp.ActiveDoc

    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)
    bOldSetting = swApp.GetUserPreferenceToggle(swExtRefUpdateCompNames)
    swApp.SetUserPreferenceToggle swExtRefUpdateCompNames, False
    Children = swRootComp.GetChildren
    ChildCount = UBound(Children)
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    For i = 0 To ChildCount
        Set swChild = Children(i)
        ' Changing component name requires component to be selected
        bRet = swChild.Select4(False, swSelData, False)
        oldName = swChild.Name2
        newName = oldName & " 123"
        swChild.Name2 = newName
    Next i
    swApp.SetUserPreferenceToggle swExtRefUpdateCompNames, bOldSetting

End Sub

Back to top

'Class1
Option Explicit

Public WithEvents swAssy As SldWorks.AssemblyDoc

Private Function swAssy_PreRenameItemNotify(ByVal EntityType As Long, ByVal oldName As String, ByVal newName As String) As Long
    Debug.Print "PRE-NOTIFICATION - about to rename component: " & oldName
End Function

Private Function swAssy_RenameItemNotify(ByVal EntityType As Long, ByVal oldName As String, ByVal newName As String) As Long
    Debug.Print "NOTIFICATION - rename component: " & newName
End Function

```

[Back to top](#)

Fire Notifications When Renaming Components Example (VB.NET)

This example shows how to fire notifications when you:

- are about to rename a component.
- rename a component.

```

-----
Preconditions:
1. Verify that these documents exist in public_documents\samples\tutorial\api:
   * beam_boltconnection.sldasm
   * beam_with_holes.sldprt
2. Open the Immediate window.
Postconditions:
1. Open public_documents\samples\tutorial\api\beam_boltconnection.sldasm.
2. Fires pre-notification before appending
   123 to each assembly component's name.
3. Fires notification when appending 123 to
   each assembly component's name.
4. Examine the FeatureManager design tree and the
   Immediate window.
NOTE: Because the assembly is used elsewhere, do
not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swAssy As AssemblyDoc

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfigMgr As ConfigurationManager
        Dim swConfig As Configuration
        Dim swRootComp As Component2
        Dim Children As Object
        Dim swChild As Component2
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim ChildCount As Integer
        Dim oldName As String
        Dim newName As String
        Dim bOldSetting As Boolean
        Dim bRet As Boolean
        Dim i As Integer
        Dim openAssem As Hashtable

        swModel = swApp.ActiveDoc
        swAssy = swModel

        ' Set up event
        swAssy = swModel
        openAssem = New Hashtable
        AttachEventHandlers()

        swConfigMgr = swModel.ConfigurationManager
        swConfig = swConfigMgr.ActiveConfiguration
        swRootComp = swConfig.GetRootComponent3(True)
        bOldSetting = swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swExtRefUpdateCompNames)
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swExtRefUpdateCompNames, False)
        Children = swRootComp.GetChildren
        ChildCount = UBound(Children)
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        For i = 0 To ChildCount
            swChild = Children(i)
            ' Changing component name requires component to be selected
            bRet = swChild.Select4(False, swSelData, False)
            oldName = swChild.Name2
            newName = oldName & " 123"
            swChild.Name2 = newName
        Next i
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swExtRefUpdateCompNames, bOldSetting)

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssy.PreRenameItemNotify, AddressOf Me.swAssy_PreRenameItemNotify
        AddHandler swAssy.RenameItemNotify, AddressOf Me.swAssy_RenameItemNotify
    End Sub

    Private Function swAssy_PreRenameItemNotify(ByVal EntityType As Integer, ByVal oldName As String, ByVal newName As String) As Integer
        Debug.Print("PRE-NOTIFICATION - about to rename component: " & oldName)
    End Function

    Private Function swAssy_RenameItemNotify(ByVal EntityType As Integer, ByVal oldName As String, ByVal newName As String) As Integer
        Debug.Print("NOTIFICATION - rename component: " & newName)
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Fire Notifications When Renaming Part Document Belonging to Assembly Example (C#)

This example shows how to fire notifications when you:

- are about to rename a part document belonging to an assembly.
- rename the part document.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\EDraw\claw\center.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Renames center to centerXXX.
// 2. Fires the PreRenameItemNotify and RenameItemNotify events.
// 3. Examine the FeatureManager design tree and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public PartDoc swPart;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Hashtable openPart = default(Hashtable);
            int errors = 0;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Set up event
            swPart = (PartDoc)swModel;
            openPart = new Hashtable();
            AttachEventHandlers();

            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("center.sldprt", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            errors = swModelDocExt.RenameDocument("centerXXX");

        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            if ((swPart != null))
            {
                swPart.PreRenameItemNotify += this.swPart_PreRenameItemNotify;
                swPart.RenameItemNotify += this.swPart_RenameItemNotify;
            }
        }

        //Fire notification when item is about to be renamed
        public int swPart_PreRenameItemNotify(int entType, string oldName, string newName)
        {
            Debug.Print("PreRenameItemNotify fired");
            return 0;
        }

        //Fire notification when item is renamed
        public int swPart_RenameItemNotify(int entType, string oldName, string newName)
        {
            Debug.Print("RenameItemNotify fired");
            return 0;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Fire Notifications When Renaming Part Document Belonging to Assembly Example (VBA)

This example shows how to fire notifications when you:

- are about to rename a part document belonging to an assembly.
- rename the part document.

```
'-----
' Preconditions:
' 1. Copy and paste Main into your project.
' 2. Insert a class module and copy and paste Class1 into that module.
' 3. Open public_documents\samples\tutorial\EDraw\claw\center.sldprt.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Renames center to centerXXX.
' 2. Fires the PreRenameItemNotify and RenameItemNotify events.
' 3. Examine the FeatureManager design tree and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swPart As SldWorks.PartDoc
Dim swPartEvents As Class1
Dim errors As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Set up event
    Set swPart = swModel
    Set swPartEvents = New Class1
    Set swPartEvents.swPart = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("center.SLDPRT", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    errors = swModelDocExt.RenameDocument("centerXXX")

End Sub

'Class1
Option Explicit

Public WithEvents swPart As SldWorks.PartDoc

Private Function swPart_PreRenameItemNotify(ByVal EntityType As Long, ByVal oldName As String, ByVal newName As String) As Long
    Debug.Print "PreRenameItemNotify fired"
End Function

'Fire notification when item is renamed
Public Function swPart_RenameItemNotify(ByVal entType As Long, ByVal oldName As String, ByVal newName As String) As Long
    Debug.Print "RenameItemNotify fired"
End Function
```

Fire Notifications When Renaming Part Document Belonging to Assembly Example (VB.NET)

This example shows how to fire notifications when you:

- are about to rename a part document belonging to an assembly.
- rename the part document.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\EDraw\claw\center.sldprt.
2. Open the Immediate window.
Postconditions:
1. Renames center to centerXXX.
2. Fires the PreRenameItemNotify and RenameItemNotify events..
3. Examine the FeatureManager design tree and Immediate window.
NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swPart As PartDoc

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim openPart As Hashtable
        Dim errors As Integer
        Dim status As Boolean

        swModel = swApp.ActiveDoc

        'Set up event
        swPart = swModel
        openPart = New Hashtable
        AttachEventHandlers()

        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("center.sldprt", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        errors = swModelDocExt.RenameDocument("centerXXX")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        If Not swPart Is Nothing Then
            AddHandler swPart.PreRenameItemNotify, AddressOf Me.swPart_PreRenameItemNotify
            AddHandler swPart.RenameItemNotify, AddressOf Me.swPart_RenameItemNotify
        End If
    End Sub

    'Fire notification when item is about to be renamed
    Public Function swPart_PreRenameItemNotify(ByVal entType As Integer, ByVal oldName As String, ByVal newName As String) As Integer
        Debug.Print("PreRenameItemNotify fired")
    End Function

    'Fire notification when item is renamed
    Public Function swPart_RenameItemNotify(ByVal entType As Integer, ByVal oldName As String, ByVal newName As String) As Integer
        Debug.Print("RenameItemNotify fired")
    End Function

End Class

```

Fire Undo and Redo Pre- and Post-notifications in Assembly Document (C#)

This example shows how to fire Undo and Redo post-notifications in an assembly document.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
//
// 1. Selects and hides the base-plate<1> component and
//    fires pre- and post-notifications indicating that an
//    Undo has occurred.
// 2. Performs a Redo and fires pre- and post-notifications.
// 3. Performs another Undo and fires pre- and post-notifications
//    so that the assembly document returns to its just-opened state.
// 4. Click OK to close each message box.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Windows.Forms;
namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssemblyDoc;
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool boolstatus = false;
            Hashtable openAssembly = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Event notification
            swAssemblyDoc = (AssemblyDoc)swModel;
            openAssembly = new Hashtable();
            AttachEventHandlers();

            // Select a component and hide it
            boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swModel.HideComponent2();
            swModel.ClearSelection2(true);

            // Undo hiding of the component
            // and fire pre- and post-notifications
            swModel.EditUndo2(1);

            // Redo hiding of the component
            // and fire pre- and post-notifications
            swModel.EditRedo2(1);

            // Undo hiding of the component
            // so that the assembly document
            // is unchanged
            swModel.EditUndo2(1);

            // Rebuild model
            swModel.ForceRebuild3(true);
        }
    }
    /// <summary>
```

```

    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }
    public void AttachSWEvents()
    {
        swAssemblyDoc.UndoPostNotify += this.swAssembly_UndoPostNotify;
        swAssemblyDoc.UndoPreNotify += this.swAssembly_UndoPreNotify;
        swAssemblyDoc.RedoPostNotify += this.swAssembly_RedoPostNotify;
        swAssemblyDoc.RedoPreNotify += this.swAssembly_RedoPreNotify;
    }
    public int swAssembly_UndoPostNotify()
    {
        //Display message after undo action occurs
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("An Undo post-notification event has been fired.");
        return 1;
    }
    public int swAssembly_UndoPreNotify()
    {
        //Display message after undo action occurs
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("An Undo pre-notification event has been fired.");
        return 1;
    }
    public int swAssembly_RedoPostNotify()
    {
        //Display message after undo action occurs
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("A Redo post-notification event has been fired.");
        return 1;
    }
    public int swAssembly_RedoPreNotify()
    {
        //Display message after undo action occurs
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("A Redo pre-notification event has been fired.");
        return 1;
    }
}
}
}

```

Fire Undo and Redo Pre- and Post-notifications in Assembly Document (VBA)

This example shows how to fire Undo and Redo pre- and post-notifications in an assembly document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
' 2. Copy and paste Main into the main module.
' 3. Click Insert > Class Module and copy and paste Class1 into that module.
'
' Postconditions:
' 1. Selects and hides the base-plate<1> component and
'    fires pre- and post-notifications indicating that an
'    Undo has occurred.
' 2. Performs a Redo and fires pre- and post-notifications.
' 3. Performs another Undo and fires pre- and post-notifications
'    so that the assembly document returns to its just-opened state.
' 4. Click OK to close each message box.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
' Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim boolstatus As Boolean
Dim swAssemblyEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Event notification
    Set swAssembly = swModel
    Set swAssemblyEvents = New Class1
    Set swAssemblyEvents.swAssembly = swApp.ActiveDoc
    ' Select a component and hide it
    boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.HideComponent2

    swModel.ClearSelection2 True

    ' Undo hiding of the component
    swModel.EditUndo2 1

    ' Redo hiding of the component
    swModel.EditRedo2 1
    ' Undo hiding of the component
    ' so that model document is unchanged
    swModel.EditUndo2 1

    ' Rebuild the model
    swModel.ForceRebuild3 True

End Sub

'Class1
Option Explicit

Public WithEvents swAssembly As SldWorks.AssemblyDoc

Private Function swAssembly_UndoPostNotify() As Long
'Display message after an Undo
    MsgBox "An Undo post-notification event has been fired."
End Function

Private Function swAssembly_UndoPreNotify() As Long
'Display message before an Undo
    MsgBox "An Undo pre-notification event has been fired."
End Function

Private Function swAssembly_RedoPostNotify() As Long
'Display message after an Redo
    MsgBox "A Redo post-notification event has been fired."
End Function

Private Function swAssembly_RedoPreNotify() As Long
'Display message before an Redo
    MsgBox "A Redo pre-notification event has been fired."
End Function
```

[Go to top](#)

[Go to top](#)

Fire Undo and Redo Pre- and Post-notifications in Assembly Document (VB.NET)

This example shows how to fire Undo and Redo pre- and post-notifications in an assembly document.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\smartcomponents\stepped_shaft.sldasm.
'
' Postconditions:
' 1. Selects and hides the base_plate<1> component and
'    fires pre- and post-notifications indicating that an
'    Undo has occurred.
' 2. Performs a Redo and fires pre- and post-notifications.
' 3. Performs another Undo and fires pre- and post-notifications
'    so that the assembly document returns to its just-opened state.
' 4. Click OK to close each message box.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Collections
Imports System.Windows.Forms

Partial Class SolidWorksMacro
    Public WithEvents swAssemblyDoc As AssemblyDoc

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim boolstatus As Boolean
        Dim openAssembly As Hashtable

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Event notification
        swAssemblyDoc = swModel
        openAssembly = New Hashtable
        AttachEventHandlers()

        ' Select a component and hide it
        boolstatus = swModelDocExt.SelectByID2("base_plate-1@stepped_shaft", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swModel.HideComponent2()
        swModel.ClearSelection2(True)

        ' Undo hiding of the component
        ' and fire pre- and post notifications
        swModel.EditUndo2(1)

        ' Redo hiding of the component
        ' and fire pre- and post-notifications
        swModel.EditRedo2(1)

        ' Undo hiding of the component
        ' so that the assembly document
        ' is unchanged
        swModel.EditUndo2(1)

        ' Rebuild model
        swModel.ForceRebuild3(True)
    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

```

```

Sub AttachSWEvents()
    AddHandler swAssemblyDoc.UndoPostNotify, AddressOf Me.swAssembly_UndoPostNotify
    AddHandler swAssemblyDoc.UndoPreNotify, AddressOf Me.swAssembly_UndoPreNotify
    AddHandler swAssemblyDoc.RedoPostNotify, AddressOf Me.swAssembly_RedoPostNotify
    AddHandler swAssemblyDoc.RedoPreNotify, AddressOf Me.swAssembly_RedoPreNotify
End Sub

```

```

Private Function swAssembly_UndoPostNotify() As Integer
    ' Display message after Undo
    ' NOTE: Because the message box may be displayed
    ' behind an opened window, you might not see it.
    ' If so, then check the Taskbar for it.
    MsgBox("An Undo post-notification event has been fired.")
End Function

```

```

Private Function swAssembly_UndoPreNotify() As Integer
    ' Display message after Undo
    ' NOTE: Because the message box may be displayed
    ' behind an opened window, you might not see it.
    ' If so, then check the Taskbar for it.
    MsgBox("An Undo pre-notification event has been fired.")
End Function

```

```

Private Function swAssembly_RedoPostNotify() As Integer
    ' Display message after Redo
    ' NOTE: Because the message box may be displayed
    ' behind an opened window, you might not see it.
    ' If so, then check the Taskbar for it.
    MsgBox("A Redo post-notification event has been fired.")
End Function

```

```

Private Function swAssembly_RedoPreNotify() As Integer
    ' Display message after Undo
    ' NOTE: Because the message box may be displayed
    ' behind an opened window, you might not see it.
    ' If so, then check the Taskbar for it.
    MsgBox("A Redo pre-notification event has been fired.")
End Function

```

```

''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```

End Class

```

Fire Undo and Redo Pre- and Post-notifications in Part Document (C#)

This example shows how to fire Undo and Redo pre- and post-notifications in a part document.

```
// -----
// Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.
//
// Postconditions:
// 1. Creates a circle, undoes it, redoes it, and undoes it again.
// 2. Fires a pre- and post-notification and displays a message box
//    before and after each Undo and Redo.
// 3. Click OK to close each message box.
//
// NOTE: Because the part is used elsewhere, do not save changes.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Collections;
using System.Windows.Forms;
namespace RedoPostNotifyPartCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPart;
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            SketchManager swSketchManager;
            SketchSegment swSketchSegment;
            bool boolstatus = false;
            Hashtable openPart;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Set up event notification
            swPart = (PartDoc)swModel;
            openPart = new Hashtable();
            AttachEventHandlers();

            // Create a circle on the
            // top face of the candlestick
            swModelDocExt = swModel.Extension;
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.00140404215739, 0.219999999999999, 0.001897848026772, false, 0,
null, 0);

            swSketchManager = swModel.SketchManager;
            swSketchSegment = swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.01296, -0.006347, 0.0);
            swModel.ClearSelection2(true);

            // Undo creation of circle
            // and fire an Undo pre- and post-
            // notification
            swModel.EditUndo2(1);

            // Redo creation of circle
            // and fire a Redo pre- and post-
            // notification
            swModel.EditRedo2(1);

            // Undo creation of circle again
            // to leave model document unchanged
            // and fire another Undo pre- and post-
```

```

        // notification
        swModel.EditUndo2(1);

        swModel.ClearSelection2(true);
        swModel.ForceRebuild3(true);
    }
    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }
    public void AttachSWEvents()
    {
        swPart.UndoPostNotify += this.swPart_UndoPostNotify;
        swPart.UndoPreNotify += this.swPart_UndoPreNotify;
        swPart.RedoPostNotify += this.swPart_RedoPostNotify;
        swPart.RedoPreNotify += this.swPart_RedoPreNotify;
    }
    public int swPart_UndoPostNotify()
    {
        // Show message after an Undo
        // Display message after Undo
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("An Undo post-notification event has been fired.");
        return 1;
    }
    public int swPart_UndoPreNotify()
    {
        // Show message after an Undo
        // Display message after Undo
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("An Undo pre-notification event has been fired.");
        return 1;
    }
    public int swPart_RedoPostNotify()
    {
        // Show message after an Undo
        // Display message after Undo
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("A Redo post-notification event has been fired.");
        return 1;
    }
    public int swPart_RedoPreNotify()
    {
        // Show message after an Undo
        // Display message after Undo
        // NOTE: Because the message box may be displayed
        // behind an opened window, you might not see it.
        // If so, then check the Taskbar for it.
        MessageBox.Show("A Redo pre-notification event has been fired.");
        return 1;
    }
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```


Fire Undo and Redo Pre- and Post-notifications in Part Document (VBA)

This example shows how to fire undo and redo pre- and post-notifications in a part document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cstick.sldprt.
' 2. Copy and paste Main into the main module.
' 3. Click Insert > Class Module and copy and paste Class1 into that module.
'
' Postconditions:
' 1. Creates a circle, undoes it, redoes it, and undoes it again.
' 2. Fires a pre- and post-notification and displays a message box
'    before and after each Undo and Redo.
' 3. Click OK to close each message box.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
'Main

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim boolstatus As Boolean
Dim swPartEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Event notification
    Set swPart = swModel
    Set swPartEvents = New Class1
    Set swPartEvents.swPart = swApp.ActiveDoc

    ' Create a circle on the
    ' top face of the candlestick
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.00140404215739, 0.219999999999999, 0.001897848026772, False, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.01296, -0.006347, 0#)

    ' Undo creation of circle
    ' and fire an undo pre - and post-
    ' notification
    swModel.EditUndo2 1

    ' Redo creation of circle
    ' and fire a redo pre- and post-
    ' notification
    swModel.EditRedo2 1

    ' Undo creation of circle again
    ' to leave model document unchanged
    ' and fire another Undo pre- and post-
    ' notification
    swModel.EditUndo2 1

    swModel.ClearSelection2 True
    swModel.ForceRebuild3 True

End Sub

'Class1
Option Explicit

Public WithEvents swPart As SldWorks.PartDoc

Private Function swPart_UndoPostNotify() As Long
    MsgBox "An Undo post-notification event has been fired."
End Function

Private Function swPart_UndoPreNotify() As Long
    MsgBox "An Undo pre-notification event has been fired."
End Function

Private Function swPart_RedoPostNotify() As Long
    MsgBox "A Redo post-notification event has been fired."
End Function

Private Function swPart_RedoPreNotify() As Long
    MsgBox "A Redo pre-notification event has been fired."
End Function
```

[Go to top](#)

[Go to top](#)

Fire Undo and Redo Pre- and Post-notifications in Part Document (VB.NET)

This example shows how to fire undo and redo pre- and post-notifications in a part document.

```

' -----
' Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.
'
' Postconditions:
' 1. Creates a circle, undoes it, redoes it, and undoes it again.
' 2. Fires a pre- and post-notification and displays a message box
'    before and after each Undo and Redo.
' 3. Click OK to close each message box.
'
' NOTE: Because the part is used elsewhere, do not save changes.
' -----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Collections

Partial Class SolidWorksMacro
    Public WithEvents swPart As PartDoc
    Public Sub Main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim boolstatus As Boolean
        Dim openPart As Hashtable

        swModel = swApp.ActiveDoc

        ' Set up event notification
        swPart = swModel
        openPart = New Hashtable
        AttachEventHandlers()

        ' Create a circle on the
        ' top face of the candlestick
        swModelDocExt = swModel.Extension
        boolstatus = swModelDocExt.SelectByID("", "FACE", 0.00140404215739, 0.219999999999999, 0.001897848026772, False, 0, Nothing,
0)

        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.01296, -0.006347, 0.0#)

        ' Undo creation of circle
        ' and fire an undo post-notification
        swModel.EditUndo2(1)

        ' Redo creation of circle
        ' and fire a redo post-notification
        swModel.EditRedo2(1)

        ' Undo creation of circle
        ' to leave model document unchanged
        ' and fire an Undo post-notification
        swModel.EditUndo2(1)

        swModel.ClearSelection2(True)
        swModel.ForceRebuild3(True)
    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub
End Class

```

End Sub

Sub AttachSWEvents()

AddHandler swPart.UndoPostNotify, AddressOf Me.swPart_UndoPostNotify
AddHandler swPart.UndoPreNotify, AddressOf Me.swPart_UndoPreNotify
AddHandler swPart.RedoPostNotify, AddressOf Me.swPart_RedoPostNotify
AddHandler swPart.RedoPreNotify, AddressOf Me.swPart_RedoPreNotify

End Sub

Function swPart_UndoPostNotify() As Integer

' Show message after an Undo
' Display message after Undo
' NOTE: Because the message box might be displayed
' behind an opened window, you might not see it.
' If so, then check the Taskbar for it.
MsgBox("An Undo post-notification event has been fired.")

End Function

Function swPart_UndoPreNotify() As Integer

' Show message after an Undo
' Display message after Undo
' NOTE: Because the message box might be displayed
' behind an opened window, you might not see it.
' If so, then check the Taskbar for it.
MsgBox("An Undo pre-notification event has been fired.")

End Function

Function swPart_RedoPostNotify() As Integer

' Show message after an Undo
' Display message after Undo
' NOTE: Because the message box might be displayed
' behind an opened window, you might not see it.
' If so, then check the Taskbar for it.
MsgBox("A Redo post-notification event has been fired.")

End Function

Function swPart_RedoPreNotify() As Integer

' Show message after an Undo
' Display message after Undo
' NOTE: Because the message box might be displayed
' behind an opened window, you might not see it.
' If so, then check the Taskbar for it.
MsgBox("A Redo pre-notification event has been fired.")

End Function

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.

''' </summary>

Public swApp As SldWorks

End Class

Flash an Add-in's Toolbar Button Example (VBA)

This example shows how to flash an add-in's CommandManager-style toolbar button, similar to what you see in many of the SOLIDWORKS Design online tutorials.

NOTE: The first example shows how to flash an add-in's CommandManager-style toolbar button using the ISwHtmlInterface interface. The second example shows how to do this using both the ISwHtmlInterface and ISldWorks interfaces.

Example 1

```
'-----  
'  
' Preconditions: SOLIDWORKS Design is running and  
'               the Fastening Feature toolbar is visible.  
'  
' Postconditions: Each time you click the Continue button in the  
'               Microsoft Visual Basic IDE, a different toolbar button on the  
'               Fastening Feature toolbar flashes and that button's  
'               tooltip is displayed.  
'  
'-----  
  
Option Explicit  
  
Dim swApp As Object  
Dim newButtonID As Long  
Dim flashCommands As Long  
Dim userID As Long  
  
Sub main()  
  
Set swApp = CreateObject("SwHtmlControl.SwHtmlInterface.14")  
For userID = 40712 To 40715 'Currently bound IDs for each button on the Fastening Feature toolbar  
    newButtonID = swApp.GetCommandID("{65F06ACE-ED12-47FA-AB16-DB6ADE842AB5}", userID)  
    flashCommands = "" + CStr(newButtonID) + ""  
    swApp.ShowBubbleTooltip newButtonID, flashCommands, 0, "", ""  
    Stop  
Next  
  
End Sub
```

Example 2

```
'-----  
'  
' Preconditions: SOLIDWORKS Design is running and  
'               the Fastening Feature toolbar is visible.  
'  
' Postconditions: Each time you click the Continue button in the  
'               Microsoft Visual Basic IDE, a different toolbar button on the  
'               Fastening Feature toolbar flashes and that button's  
'               tooltip is displayed.  
'  
'-----  
  
Option Explicit  
  
Dim swHTMLInt As Object  
Dim swApp As Object  
Dim newButtonID As Long  
Dim flashCommands As Long  
Dim userID As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swHTMLInt = CreateObject("SwHtmlControl.SwHtmlInterface.14")  
  
For userID = 40712 To 40715 'Currently bound IDs for each button on the Fastening Feature toolbar
```

```
newButtonID = swHTMLInt.GetCommandID("{65F06ACE-ED12-47FA-AB16-DB6ADE842AB5}", userId)
```

```
flashCommands = "" + CStr(newButtonID) + ""
```

```
swApp.ShowBubbleTooltip newButtonID, flashCommands, 0, "", ""
```

```
Stop
```

```
Next
```

```
End Sub
```

Flatten Sheet Metal Part Example (VBA)

This example shows how to flatten a sheet metal part.

```
' -----  
' Preconditions:  
' 1. Open a sheet metal part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Flattens the sheet metal part.  
' 2. Examine the graphics area and Immediate window.  
' -----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim nBendState As Long  
    Dim nRetVal As Long  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    nBendState = swModel.GetBendState  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "    Original bend state = " & nBendState  
  
    If nBendState <> swSMBendStateFlattened Then  
        nRetVal = swModel.SetBendState(swSMBendStateFlattened)  
        Debug.Print "    Modified bend state = " & nRetVal  
  
        ' Rebuild to see changes  
        bRet = swModel.EditRebuild3: Debug.Assert bRet  
    End If  
  
End Sub
```

Flip Dowel Pin Symbol Example (C#)

This example shows how to insert and flip a dowel pin symbol in a drawing.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document
//    to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Selects a circular edge in a drawing view.
// 3. Inserts a dowel pin symbol.
// 4. Selects the dowel pin symbol and flips it.
// 5. Examine the drawing and the Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DowelSymbolCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            DowelSymbol swDowelSymbol = default(DowelSymbol);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Annotation swAnnotation = default(Annotation);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open drawing document and insert a dowel pin symbol
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem20.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "EDGE", 0.128048002364532, 0.165546371003625, -1499.96487716824, false, 0, null, 0);
            swDrawingDoc = (DrawingDoc)swModel;
            swDowelSymbol = (DowelSymbol)swDrawingDoc.InsertDowelSymbol();
            swModel.ClearSelection2(true);

            //Flip the dowel pin symbol
            status = swModelDocExt.SelectByID2("DetailItem354@Drawing View1", "DOWELSYM", 0.121630029714286, 0.180965058285714, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swDowelSymbol = (DowelSymbol)swSelectionMgr.GetSelectedObject6(1, -1);
            swDowelSymbol.Flipped = true;
            swModel.EditRebuild3();
            Debug.Print("Dowel pin symbol flipped? " + swDowelSymbol.Flipped);
            swAnnotation = (Annotation)swDowelSymbol.GetAnnotation();
            Debug.Print("Name of dowel pin symbol annotation: " + swAnnotation.GetName());
        }
    }
    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Flip Dowel Pin Symbol Example (VBA)

This example shows how to insert and flip a dowel pin symbol in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document
'    to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Selects a circular edge in a drawing view.
' 3. Inserts a dowel pin symbol.
' 4. Selects the dowel pin symbol and flips it.
' 5. Examine the drawing and the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swDowelSymbol As SldWorks.DowelSymbol
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swAnnotation As SldWorks.Annotation
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open drawing document and insert a dowel pin symbol
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "EDGE", 0.128048002364532, 0.165546371003625, -1499.96487716824, False, 0, Nothing, 0)
    Set swDrawingDoc = swModel
    Set swDowelSymbol = swDrawingDoc.InsertDowelSymbol()
    swModel.ClearSelection2 True

    'Flip the dowel pin symbol
    status = swModelDocExt.SelectByID("DetailItem354@Drawing View1", "DOWELSYM", 0.121630029714286, 0.180965058285714, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swDowelSymbol = swSelectionMgr.GetSelectedObject6(1, -1)
    swDowelSymbol.Flipped = True
    swModel.EditRebuild3
    Debug.Print "Dowel pin symbol flipped? " & swDowelSymbol.Flipped
    Set swAnnotation = swDowelSymbol.GetAnnotation
    Debug.Print "Name of dowel pin symbol annotation: " & swAnnotation.GetName

End Sub
```

Flip Dowel Pin Symbol Example (VB.NET)

This example shows how to insert and flip a dowel pin symbol in a drawing.

```

-----
Preconditions:
1. Verify that the specified drawing document
   to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified drawing document.
2. Selects a circular edge in a drawing view.
3. Inserts a dowel pin symbol.
4. Selects the dowel pin symbol and flips it.
5. Examine the drawing and the Immediate window.
NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawingDoc As DrawingDoc
        Dim swDowelSymbol As DowelSymbol
        Dim swSelectionMgr As SelectionMgr
        Dim swAnnotation As Annotation
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open drawing document and insert a dowel pin symbol
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "EDGE", 0.128048002364532, 0.165546371003625, -1499.96487716824, False, 0, Nothing, 0)
        swDrawingDoc = swModel
        swDowelSymbol = swDrawingDoc.InsertDowelSymbol()
        swModel.ClearSelection2(True)

        'Flip the dowel pin symbol
        status = swModelDocExt.SelectByID("DetailItem354@Drawing View1", "DOWELSYM", 0.121630029714286, 0.180965058285714, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swDowelSymbol = swSelectionMgr.GetSelectedObject6(1, -1)
        swDowelSymbol.Flipped = True
        swModel.EditRebuild3()
        Debug.Print("Dowel pin symbol flipped? " & swDowelSymbol.Flipped)
        swAnnotation = swDowelSymbol.GetAnnotation
        Debug.Print("Name of dowel pin symbol annotation: " & swAnnotation.GetName)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Flip Sketch Example (C#)

This example shows how to flip a sketch about a coordinate system axis.

```
//-----  
// Preconditions: Open a model document and select a sketch.  
//  
// Postconditions: Flips the sketch about the x axis.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace FlipSketch_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            swModel.SketchModifyFlip(1);  
  
        }  
        public SldWorks swApp;  
    }  
}
```

Flip Sketch Example (VBA)

This example shows how to flip a sketch about a coordinate system axis.

```
'-----  
' Preconditions: Open a model document and select a sketch.  
'  
' Postconditions: Flips the sketch about the x axis.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    swModel.SketchModifyFlip 1
```

End Sub

Flip Sketch Example (VB.NET)

This example shows how to flip a sketch about a coordinate system axis.

```
'-----  
' Preconditions: Open a model document and select a sketch.  
'-----  
' Postconditions: Flips the sketch about the x axis.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2  
        swModel = swApp.ActiveDoc  
  
        swModel.SketchModifyFlip(1)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Flip Sketch Picture Example (C#)

This example shows how to flip a sketch picture.

```
//-----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Copy an image file (i.e., .bmp, .gif, .jpg, .jpeg, .tif, .wmf) to
//    c:\temp.
// 3. Replace image_file in the ISketchManager::InsertSketchPicture parameter
//    with the name of the copied file.
// 4. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document.
// 2. Creates Sketch1 and Sketch Picture1 in the graphics area and the
//    FeatureManager design tree.
// 3. Selects Sketch Picture1 and flips it top to bottom.
// 4. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace FlipSketchPicture_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature swFeat;
        SketchPicture swSketchPicture;
        SelectionMgr swSelMgr;
        bool boolstatus;
        double width;
        double height;
        double angle;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2020\\templates\\Part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModel.SketchManager.InsertSketch(true);
            swSketchPicture = swModel.SketchManager.InsertSketchPicture2("c:\\temp\\image_file", false);
            swModel.SketchManager.InsertSketch(true);

            boolstatus = swModel.Extension.SelectByID("Sketch Picture1", "SKETCHBITMAP", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

            Debug.Print("Feature name = " + swFeat.Name);

            boolstatus = swSketchPicture.Flip(false);
            Debug.Print(" Sketch picture flipped? " + swSketchPicture.Flipped);

            swSketchPicture.GetSize(ref width, ref height);
            Debug.Print(" Width: " + width * 1000 + " mm");
            Debug.Print(" Height: " + height * 1000 + " mm");

            angle = swSketchPicture.Angle;
            //1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print(" Angle: " + angle * 57.3 + " degrees");
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Flip Sketch Picture Example (VBA)

This example shows how to flip a sketch picture.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Copy an image file (i.e., .bmp, .gif, .jpg, .jpeg, .tif, .wmf) to
'    c:\temp.
' 3. Replace image_file in the ISketchManager::InsertSketchPicture parameter
'    with the name of the copied file.
' 4. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document.
' 2. Creates Sketch1 and Sketch Picture1 in the graphics area and the
'    FeatureManager design tree.
' 3. Selects Sketch Picture1 and flips it top to bottom.
' 4. Inspect the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swSketchPicture As SldWorks.SketchPicture
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim width As Double
Dim height As Double
Dim angle As Double

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2020\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swApp.ActiveDoc

    Set swSelMgr = swModel.SelectionManager
    swModel.SketchManager.InsertSketch True
    Set swSketchPicture = swModel.SketchManager.InsertSketchPicture2("c:\temp\image_file", False)
    swModel.SketchManager.InsertSketch True

    boolstatus = swModel.Extension.SelectByID2("Sketch Picture1", "SKETCHBITMAP", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)

    Debug.Print "Feature name = " & swFeat.Name

    boolstatus = swSketchPicture.Flip(False)
    Debug.Print " Sketch picture flipped? " & swSketchPicture.Flipped

    swSketchPicture.GetSize width, height
    Debug.Print " Width: " & width * 1000 & " mm"
    Debug.Print " Height: " & height * 1000 & " mm"

    angle = swSketchPicture.Angle
    '1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Debug.Print " Angle: " & angle * 57.3 & " degrees"
End Sub

```

Flip Sketch Picture Example (VB.NET)

This example shows how to flip a sketch picture.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Copy an image file (i.e., .bmp, .gif, .jpg, .jpeg, .tif, .wmf) to
'    c:\temp.
' 3. Replace image_file in the ISketchManager::InsertSketchPicture parameter
'    with the name of the copied file.
' 4. Open an Immediate window.
'-----
' Postconditions:
' 1. Creates a new model document.
' 2. Creates Sketch1 and Sketch Picture1 in the graphics area and the
'    FeatureManager design tree.
' 3. Selects Sketch Picture1 and flips it top to bottom.
' 4. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swFeat As Feature
    Dim swSketchPicture As SketchPicture
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean
    Dim width As Double
    Dim height As Double
    Dim x As Double
    Dim y As Double
    Dim angle As Double

    Sub main()

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2020\templates\Part.prtdot", 0, 0, 0)
        swModel = swApp.ActiveDoc

        swSelMgr = swModel.SelectionManager
        swModel.SketchManager.InsertSketch(True)
        swSketchPicture = swModel.SketchManager.InsertSketchPicture2("c:\temp\image_file", False)
        swModel.SketchManager.InsertSketch(True)

        boolstatus = swModel.Extension.SelectByID2("Sketch Picture1", "SKETCHBITMAP", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)

        Debug.Print("Feature name = " & swFeat.Name)

        boolstatus = swSketchPicture.Flip(False)
        Debug.Print(" Sketch picture flipped? " & swSketchPicture.Flipped)

        swSketchPicture.GetSize(width, height)
        Debug.Print(" Width: " & width * 1000 & " mm")
        Debug.Print(" Height: " & height * 1000 & " mm")

        angle = swSketchPicture.Angle
        '1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print(" Angle: " & angle * 57.3 & " degrees")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Force Rebuild Example (VBA)

This example shows how to force a rebuild of a model.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Rebuilds the model.
' 2. Examine the Immediate window.
'
' NOTE: This macro turns off saving the model automatically. Before
' running this macro, click Tools > Options >
' System Options > Backup/Recover > Save auto-recover information
' every. If saving the model automatically before running this
' macro, then change this option back to its original values
' after running the macro.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim nStart As Single
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Turn off saving model automatically
    swApp.SetUserPreferenceIntegerValue swAutoSaveInterval, False

    ' Use VBA Timer function to calculate time to rebuild model
    nStart = Timer

    bRet = swModel.ForceRebuild3(False)

    Debug.Print "File = " + swModel.GetPathName
    Debug.Print " Time to rebuild model = " & Timer - nStart & " sec"

End Sub
```

Forcibly Rebuild All Configurations Example (VBA)

This example shows how to forcibly rebuild all of the configurations in a SOLIDWORKS Design part or assembly document.

```
'-----
' Postconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Rebuilds each configuration in the the part or
'   assembly.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim vConfNameArr As Variant
    Dim sConfigName As String
    Dim start As Single
    Dim i As Long
    Dim bShowConfig As Boolean
    Dim bRebuild As Boolean
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc

    Debug.Print "File = " + swModel.GetPathName

    vConfNameArr = swModel.GetConfigurationNames
    For i = 0 To UBound(vConfNameArr)
        sConfigName = vConfNameArr(i)
        bShowConfig = swModel.ShowConfiguration2(sConfigName)
        start = Timer
        bRebuild = swModel.ForceRebuild3(False)
        Debug.Print " Configuration = " & sConfigName
        Debug.Print " ShowConfig = " & bShowConfig
        Debug.Print " Rebuild = " & bRebuild
        Debug.Print " Time = " & Timer - start & " seconds"
    Next i

End Sub
```

Fully Define Under Defined Sketch Example (C#)

This example shows how to fully define an under defined sketch.

```
//-----
// Preconditions: Open a part document containing an under defined sketch.
//
// Postconditions:
// 1. Fully defines the under defined sketch.
// 2. Open the sketch to verify.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace FullyDefineSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeature = default(Feature);
            bool bValue = false;
            SketchManager swSketchManager = default(SketchManager);
            ModelDocExtension swModelExtension = default(ModelDocExtension);
            int lStatus = 0;
            int lMarkHorizontal = 0;
            int lMarkVertical = 0;
            SelectionMgr swSelectionManager = default(SelectionMgr);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelExtension = swModel.Extension;
            swSketchManager = swModel.SketchManager;
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;

            swModel.ClearSelection2(true);

            // These are the marks expected for the dimension datums
            lMarkHorizontal = 2;
            lMarkVertical = 4;

            swFeature = (Feature)swModel.FirstFeature();

            while ((swFeature != null))
            {
                if ((swFeature.GetTypeName() == "ProfileFeature"))
                {
                    break;
                }
                swFeature = (Feature)swFeature.GetNextFeature();
            }

            if ((swFeature != null))
            {
                bValue = swFeature.Select2(false, 0);
                swSketchManager.InsertSketch(false);

                // OR together the marks for the vertical and horizontal datums;
                // You cannot select the same point twice with different marks
                bValue = swModelExtension.SelectByID("Point1@Origin", "EXTSKETCHPOINT", 0, 0, 0, false, lMarkHorizontal |
lMarkVertical, null, 0);
                lStatus = swSketchManager.FullyDefineSketch(true, true,
(int)swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical |
(int)swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, true, 1, null, 1, null, 1, 1);

                swSketchManager.InsertSketch(true);
            }
        }

        public SldWorks swApp;
    }
}
```

Fully Define Under Defined Sketch Example (VBA)

This example shows how to fully define an under defined sketch.

```
'-----
' Preconditions: Open a part document containing an under defined sketch.
'
' Postconditions:
' 1. Fully defines the under defined sketch.
' 2. Open the sketch to verify.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Dim swModel As SldWorks.ModelDoc2
    Dim swFeature As SldWorks.Feature
    Dim bValue As Boolean
    Dim swSketchManager As SldWorks.SketchManager
    Dim swModelExtension As SldWorks.ModelDocExtension
    Dim lStatus As Long
    Dim lMarkHorizontal As Long
    Dim lMarkVertical As Long
    Dim swSelectionManager As SldWorks.SelectionMgr

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelExtension = swModel.Extension
    Set swSketchManager = swModel.SketchManager
    Set swSelectionManager = swModel.SelectionManager

    swModel.ClearSelection2 True

    ' These are the marks expected for the dimension datums
    lMarkHorizontal = 2
    lMarkVertical = 4

    Set swFeature = swModel.FirstFeature
    Do While (Not (swFeature Is Nothing))
        If (swFeature.GetTypeName = "ProfileFeature") Then
            Exit Do
        End If
        Set swFeature = swFeature.GetNextFeature
    Loop

    If (Not (swFeature Is Nothing)) Then
        bValue = swFeature.Select2(False, 0)
        swSketchManager.InsertSketch False
        ' OR together the marks for the vertical and horizontal datums;
        ' You cannot select the same point twice with different marks
        bValue = swModelExtension.SelectByID("Point1@Origin", "EXTSKETCHPOINT", 0, 0, 0, False, lMarkHorizontal Or lMarkVertical, Nothing, 0)
        lStatus = swSketchManager.FullyDefineSketch(True, True, swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical Or
swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, True, 1, Nothing, 1, Nothing, 1, 1)
        swSketchManager.InsertSketch True
    End If

End Sub
```

Fully Define Under Defined Sketch Example (VB.NET)

This example shows how to fully define an under defined sketch.

```
'-----
' Preconditions: Open a part document containing an under defined sketch.
'
' Postconditions:
' 1. Fully defines the under defined sketch.
' 2. Open the sketch to verify.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swFeature As Feature
        Dim bValue As Boolean
        Dim swSketchManager As SketchManager
        Dim swModelExtension As ModelDocExtension
        Dim lStatus As Integer
        Dim lMarkHorizontal As Integer
        Dim lMarkVertical As Integer
        Dim swSelectionManager As SelectionMgr

        swModel = swApp.ActiveDoc
        swModelExtension = swModel.Extension
        swSketchManager = swModel.SketchManager
        swSelectionManager = swModel.SelectionManager

        swModel.ClearSelection2(True)

        ' These are the marks expected for
        the dimension datums

        lMarkHorizontal = 2
        lMarkVertical = 4

        swFeature = swModel.FirstFeature

        Do While (Not (swFeature Is Nothing))

            If (swFeature.GetTypeName = "ProfileFeature") Then

                Exit Do

            End If

            swFeature = swFeature.GetNextFeature

        Loop

        If (Not (swFeature Is Nothing)) Then

            bValue = swFeature.Select2(False,

0)

            swSketchManager.InsertSketch(False)

            ' OR together the marks for the
            vertical and horizontal datums;
```

```

        '
        You cannot select the same point twice with different marks

        bValue = swModelExtension.SelectByID2("Point1@Origin", "EXTSKETCHPOINT",
        0, 0, 0, False, lMarkHorizontal Or lMarkVertical, Nothing,
        0)
        lStatus = swSketchManager.FullyDefineSketch(True, True, swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Vertical Or
        swSketchFullyDefineRelationType_e.swSketchFullyDefineRelationType_Horizontal, True, 1, Nothing, 1, Nothing, 1,
        1)

        swSketchManager.InsertSketch(True)

    End If

End Sub

Public swApp As SldWorks

End Class

```

Functional Categories

[Send Feedback](#)

This topic lists the SOLIDWORKS interfaces categorized by functionality.

- [Application Interfaces](#)
- [Annotation Interfaces](#)
- [Assembly Interfaces](#)
- [Drawing Interfaces](#)
- [Configuration Interfaces](#)
- [DimXpert Interfaces](#)
- [Enumeration Interfaces](#)
- [Feature Interfaces](#)
- [Model Interfaces](#)
- [Motion Studies Interfaces](#)
- [Sketch Interfaces](#)
- [Utility Interfaces](#)
- [User-interface Interfaces](#)
- [Custom Interfaces](#)

NOTE: Interfaces and delegates appear in alphabetical order on the Contents tab in their respective interop assembly namespace book.

Application Interfaces

- [IAssemblyDoc](#)
- [IDrawingDoc](#)
- [IModelDoc2](#)
- [IModelDocExtension](#)
- [IPartDoc](#)
- [ISldWorks](#)

[Back to top](#)

Annotation Interfaces

- [Annotation](#)
- [AutoBackupOptions](#)
- [BallroomOptions](#)
- [BallroomSize](#)
- [CenterAngle](#)
- [CenterMark](#)
- [CThread](#)
- [CustomOption](#)
- [DatumTag](#)
- [DatumTagSizeSym](#)
- [DimensionExceedance](#)
- [DisplayDimension](#)
- [IDownSymbol](#)
- [IPlot](#)
- [IPlotFrame](#)
- [IMCFrameData](#)
- [MailBookender](#)
- [Note](#)
- [Parameters](#)
- [PMICFrameData](#)
- [PMICFrameFeature](#)
- [PMICDatumTarget](#)
- [PMICFrameData](#)
- [PMICDimensionOption](#)
- [PMICFrameData](#)
- [PMICFrameData](#)
- [PMICToolData](#)
- [PMICFrameDatum](#)
- [RevisionLog](#)
- [TSymbol](#)
- [SketchBlockDefinition](#)
- [SketchBlockInquire](#)
- [SketchBlockDefinition](#)
- [Table Annotations](#)
 - [BlendTableAnnotation](#)
 - [BlendTableAnnotation](#)
 - [GammaTableAnnotation](#)
 - [GeneralTableAnnotation](#)
 - [GeneralToleranceTableAnnotation](#)
 - [HatchTableAnnotation](#)
 - [PunchTableAnnotation](#)
 - [RevisionTableAnnotation](#)
 - [SectionAnnotation](#)
 - [FileBlockTableAnnotation](#)
 - [WeldmentCutListAnnotation](#)
 - [Weldment](#)
 - [WeldSymbol](#)

[Back to top](#)

Assembly Interfaces

- [AdvancedSelectionCriteria](#)
- [Collision](#)
- [CollisionDetectionGroup](#)
- [CollisionDetectionManager](#)
- [Component2](#)
- [ConvexPolyhedron](#)
- [ExcludeStep](#)
- [GroupOfInterFeatureData](#)
- [Interference](#)
- [InterferenceDetectionDir](#)
- [InterferComponentFeatureData](#)
- Mater**
 - [AInodeMaterialFeatureData](#)
 - [BInodeMaterialFeatureData](#)
 - [ColorMaterialFeatureData](#)
 - [ConcentricMaterialFeatureData](#)
 - [ElasticMaterialFeatureData](#)
 - [GrainMaterialFeatureData](#)
 - [HingeMaterialFeatureData](#)
 - [InodeMaterialFeatureData](#)
 - [InterCompositeMaterialFeatureData](#)
 - [LockMaterialFeatureData](#)
 - [Material2](#)
 - [Materiality2](#)
 - [MaterialFeatureData](#)
 - [MaterialReference](#)
 - [MaterialReference](#)
 - [MaterialReferenceData](#)
 - [MaterialReferenceMaterialFeatureData](#)
 - [ProfileInterMaterialFeatureData](#)
 - [RigidPlasticMaterialFeatureData](#)
 - [ScrewMaterialFeatureData](#)
 - [SkinMaterialFeatureData](#)
 - [SymmetricMaterialFeatureData](#)
 - [ThermalMaterialFeatureData](#)
 - [WeldMaterialFeatureData](#)

[Back to top](#)

Drawing Interfaces

- BreakLine
- CenterOfMass
- DrawOrder
- DrawComponent
- Direction
- FaceType
- Layers
 - Layer
 - LayerView
- ProjectionArrow
- Sheet
- SheetCutterEdge
- Tables
 - BlendTable
 - BoreFeature
 - BoomTable
 - FrameTableFeature
 - GeneralLabelFeature
 - GeneralToleranceTableFeature
 - HoleTable
 - InsertTable
 - RevisionTableFeature
 - TimeBlockTableFeature
 - TableArrow
- TitleBlock
- View
- ViewDoc

[Back to top](#)

Configuration Interfaces

- [IConfiguration](#)
- [IConfigurationManager](#)
- [IDesignTime](#)

- [Back to top](#)

150

- [Back to top](#)

IDok

- [CosmeticThreadFeatureData](#)
- [CosmeticWebReadFeatureData](#)
- [DeleteBodyFeatureData](#)
- [DeleteFeatureDataOn](#)
- [DomainFeatureData](#)
- [DraftFeatureData2](#)
- [ExtrudeFeatureData2](#)
- [FeatureStatistics](#)
- [FeatureSketchFeatureData](#)
- [IndirectFeatureData](#)
- [IntersectFeatureData](#)
- [JoinFeatureData](#)
- [JoinFeatureData2](#)
- [LibraryFormJoinFeatureData](#)
- [LiftOffFeatureData](#)
- [MacroFeatureData](#)
- [MacroSketchBodyFeatureData](#)
- [MirroredFeatureData](#)
- [MirroredFeatureData2](#)
- [RibFeatureData](#)
- [SaveBodyFeatureData](#)
- [SketchFeatureData](#)
- [SketchFeatureData2](#)
- [SketchPicture](#)
- [SplitBodyFeatureData](#)
- [SplitFeatureData](#)
- [TabAndSketchFeatureData](#)
- [UnlaminatedSketchGroupData](#)
- [WrapSketchFeatureData](#)

[Back to top](#)

Model Interfaces

- [IAnnotationView](#)
- [IAttribute](#)
- [IAttributeDef](#)
- [IBody2](#)
- [ICoEdge](#)
- [ICurve](#)
- [IDimension](#)
- [IEdge](#)
- [IEdgePoint](#)
- [IEntity](#)
- [IFace2](#)
- [IFacet](#)
- [IGraphicsBody](#)
- [ILoop2](#)
- [IMeshBody](#)
- [IMidSurface3](#)
- [IModeler](#)
- [IParameter](#)
- [ISectionViewData](#)
- [ISpring](#)
- [ISurface](#)
- [IVertex](#)

[Back to top](#)

Motion Studies

- [IAVPParameter](#)
- [CosmosMotionStudyProperties](#)
- [CosmosMotionStudyResults](#)
- [MotionPlotAxis-featureData](#)
- [MotionPlot-featureData](#)
- [MotionPlot-featureOutput](#)
- [MotionStudy](#)
- [MotionStudyManager](#)
- [MotionStudyProperties](#)
- [MotionStudyResults](#)
- [PhysicalSimulationMotionStudyProperties](#)
- [SimulationFeatureData](#)
- [SimulationJumper-featureData](#)
- [SimulationForceFeatureData](#)
- [SimulationGravityFeatureData](#)
- [SimulationMotor-featureData](#)
- [SimulationSpring-featureData](#)

[Back to top](#)

Sketch

- [ISketch](#)
- [ISketchArc](#)
- [ISketchBlockDefinition](#)
- [ISketchBlockInstance](#)
- [ISketchContour](#)
- [ISketchEllipse](#)
- [ISketchHatch](#)
- [ISketchLine](#)
- [ISketchManager](#)
- [ISketchParabola](#)
- [ISketchPath](#)
- [ISketchPoint](#)
- [ISketchRegion](#)
- [ISketchRelation](#)
- [ISketchRelationManager](#)
- [ISketchSegment](#)
- [ISketchSlot](#)
- [ISketchSpline](#)
- [ISketchText](#)
- [SplineHandle](#)

[Back to top](#)

Utility Interfaces

- [IBM/TableSort/Data](#)
- [IScriptParamData](#)
- [ISurferPage](#)
- [IsurfacePage](#)
- [CustomPropertyManager](#)
- [Kali](#)
- [KeyboardOptions](#)
- [LanguageResult](#)
- [LanguageSpecification](#)
- [LicenseSpecification](#)
- [Environment](#)
- [Equation](#)
- [ExceptionData](#)
- [FileEntity](#)
- [FormTemplate](#)
- [HoleStatementData](#)
- [Import3DInterconnectedData](#)
- [Import3DSchema](#)
- [Import3DShape](#)
- [ImportStepData](#)
- [ImageInfo](#)
- [MassProcess2](#)
- [MassPropertyOverrideOptions](#)
- [MathTransform](#)
- [Materiality](#)
- [MatrixAccess](#)
- [Measure](#)
- [PanelSetup](#)
- [PdmObjectSpecification](#)
- [PrintSpecification](#)
- [SafeArrayUtility](#)
- [SaveAsTemplate](#)
- [SessionExperienceOptions](#)
- [Scam3D](#)
- [SelectedData](#)
- [SelectionData](#)
- [SliceData](#)
- [SOLIDWORKS Design Checker](#)
- [SurfaceParameterizationData](#)
- [SurfaceFChains](#)
- [Translation](#)
- [TextFormat](#)
- [Texture](#)
- [Test Plan](#)

[Back to top](#)

User-interface Interfaces

- **Appearances and Decals**
 - [IAppearanceSetting](#)
 - [IText](#)
 - [ISpRBRMaterial](#)
 - [ISurfaceDecalProperties](#)
 - [IRayTraceHandler](#)
 - [IRayTraceRendererOptions](#)
 - [IRenderMaterial](#)
- **ICallout**
- **CommandManager**
 - [ICCommandGroup](#)
 - [ICCommandManager](#)
 - [ICCommandTab](#)
 - [ICCommandTabBox](#)
 - [ICommandGroup](#)
- **IDisplayStateSetting**
- **IDrapAreaManipulator**
- **IFeatureView**
- **IFrame**
- **ILightDialog**

- [MaterialVisualPropertiesData](#)
- [MessageBarDefinition](#)
- [ModelView](#)
- [ModelViewManager](#)
- [ModelWindow](#)
- [Mouse](#)
- [PackAndGo](#)
- [PlaneManager](#)
- [PrintJobDialog](#)
- [PropertyManager](#)
 - [PropertyManagerPage2](#)
 - [PropertyManagerPageActiveX](#)
 - [PropertyManagerPageBitmap](#)
 - [PropertyManagerPageBitmapButton](#)
 - [PropertyManagerPageButton](#)
 - [PropertyManagerPageCheckbox](#)
 - [PropertyManagerPageComboBox](#)
 - [PropertyManagerPageColor](#)
 - [PropertyManagerPageGroup](#)
 - [PropertyManagerPageLabel](#)
 - [PropertyManagerPageListbox](#)
 - [PropertyManagerPageNumberbox](#)
 - [PropertyManagerPageOption](#)
 - [PropertyManagerPageSelectionbox](#)
 - [PropertyManagerPageSlider](#)
 - [PropertyManagerPageTab](#)
 - [PropertyManagerPageTextbox](#)
 - [PropertyManagerPageWindowFromHandle](#)
- [RenamedDocumentsReferences](#)
- [Selection Sets](#)
 - [SelectionSet](#)
 - [SelectionSetFolder](#)
 - [SelectionSetItem](#)
- [Sensors](#)
 - [DimensionSensorData](#)
 - [Sensor](#)
- [SnapShot](#)
- [SOLIDWORKS MBD 3D PDF](#)
 - [MBD3DPdfData](#)
 - [TextAndCustomProperty](#)
- [ShapeBarPage](#)
- [SWHtmlInterface](#)
- [SWPropertySheet](#)
- [TemplateView](#)
- [TreeControlItem](#)
- [TreeManipulator](#)
- [UserNotificationDefinition](#)
- [UserNotification](#)

[Back to top](#)

Custom Interfaces

- [IMessageBarHandler](#)
- [IPropertyManagerPage2Handler9](#)
- [ISWCPrinter](#)
- [ISWCustomHandler](#)
- [ISWAddIn](#)
- [ISWCustomEncoder](#)
- [ISWColorControl1](#)
- [ISWColorFeature](#)
- [ISWManipulationHandler2](#)
- [ISWFeatureAction](#)
- [ISWPFManager](#)
- [ISWPFPrint](#)
- [ISWBackUp](#)
- [ISWScene](#)
- [ISWPrintAPIHandler](#)
- [UserNotificationHandler](#)

[Back to top](#)

Get Active Document Dependents Example (VBA)

This example shows how to return all of the dependent files for the currently active document. You can run this program on any type of SOLIDWORKS Design document.

```
'-----  
Option Explicit  
Sub main()  
    Dim swApp           As SldWorks.SldWorks  
    Dim swModel          As SldWorks.ModelDoc2  
    Dim vDepend          As Variant  
    Dim i                As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    vDepend = swModel.GetDependencies2(True, True, True)  
  
    Debug.Print swModel.GetPathName  
    For i = 0 To (UBound(vDepend) - 1) / 2  
        Debug.Print "      " + vDepend(2 * i) + " --> " + vDepend(2 * i + 1)  
    Next i  
End Sub  
'-----
```

Get Active PropertyManager Page Name Example (C#)

This example shows how to get the name of the active PropertyManager page.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.
// 2. In SOLIDWORKS Design, click Insert > Features > Fillet/Round.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets the name of the active PropertyManager page.
// 2. Examine the Immediate window.
// 3. In the SOLIDWORKS Design user-interface:
//    a. Close the PropertyManager page.
//    b. Close the part document.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            string pageName = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Print the name of the active PropertyManager page in the Immediate window
            pageName = swModelDocExt.GetActivePropertyManagerPage();

            Debug.Print("Name of active PropertyManager page: " + pageName);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Active PropertyManager Page Name Example (VBA)

This example shows how to get the name of the active PropertyManager page.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. At Stop:
'   a. In the SOLIDWORKS Design user-interface,
'      click Insert > Features > Fillet/Round.
'   b. In the IDE, click Continue.
' 2. Gets the name of the active PropertyManager page.
' 4. Examine the Immediate window.
' 5. In the SOLIDWORKS Design user-interface:
'   a. Close the PropertyManager page.
'   b. Close the part document.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim file As String
Dim errors As Long
Dim warnings As Long
Dim pageName As String

Sub main()

Set swApp = Application.SldWorks
file = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
Set swModel = swApp.OpenDoc6(file, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension

Stop
' 1. Click Insert > Features > Fillet/Round in the SOLIDWORKS user-interface
' 2. Click Continue in the IDE

' Print the name of the active PropertyManager page in the Immediate window
pageName = swModelDocExt.GetActivePropertyManagerPage
Debug.Print "Name of active PropertyManager page: " & pageName

End Sub
```

Get Active PropertyManager Page Name Example (VB.NET)

This example shows how to get the name of the active PropertyManager page.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.
' 2. In SOLIDWORKS Design, click Insert > Features > Fillet/Round.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the name of the active PropertyManager page.
' 2. Examine the Immediate window.
' 3. In the SOLIDWORKS Design user-interface:
'   a. Close the PropertyManager page.
'   b. Close the part document.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim pageName As String

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Print the name of the active PropertyManager page in the Immediate window
        pageName = swModelDocExt.GetActivePropertyManagerPage
        Debug.Print("Name of active PropertyManager page: " & pageName)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Advanced Properties of Spot Light Example (VBA)

This example shows how to get the attenuation-related, advanced properties of a specified SOLIDWORKS Design spot light.

```
'-----
' Preconditions:
' 1. Open a part that has a spot light
'   whose SOLIDWORKS Design-internal light source
'   name is SW#3.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets advanced properties of SOLIDWORKS Design-internal
'   light source SW#3.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim sLightName As String
Dim lLightCount As Long, i As Long
Dim dExponent As Double, dAttenuationConst As Double, dAttenuationLinear As Double, dAttenuationQuad As Double

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    lLightCount = swModel.GetLightSourceCount
    Debug.Print "Number of lights: " & lLightCount
    For i = 0 To lLightCount - 1
        ' Get the SOLIDWORKS-internal light source name
        sLightName = swModel.GetLightSourceName(i)
        Debug.Print "SOLIDWORKS-internal light source name: " & sLightName
        If sLightName = "SW#3" Then
            swModelDocExt.GetAdvancedSpotLightProperties sLightName, dExponent, dAttenuationConst, dAttenuationLinear, dAttenuationQuad
            Debug.Print " Attenuation exponent: " & dExponent
            Debug.Print " Attenuation constant factor: " & dAttenuationConst
            Debug.Print " Attenuation linear factor: " & dAttenuationLinear
            Debug.Print " Attenuation quadratic factor: " & dAttenuationQuad
        End If
    Next i
End Sub
```

Get All Edges in Visible Component in Drawing View Example (VBA)

This method gets all of the edges in the first visible component in a drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing of an assembly with multiple
'    components and a drawing view named Drawing View1.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets all of the visible components in the assembly in
'    Drawing View1.
' 2. Prints the name of the first visible component to the
'    Immediate window.
' 3. Iterates and selects all edges on the first visible
'    component.
' 4. Examine the Immediate window and drawing.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim swComp As SldWorks.Component2
Dim swView As SldWorks.View
Dim swEnt As SldWorks.Entity
Dim vComps As Variant
Dim vEdges As Variant
Dim itr As Long
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    boolstatus = swModel.ActivateView("Drawing View1")
    boolstatus = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    swModel.ClearSelection2 True

    'Get all visible components
    vComps = swView.GetVisibleComponents

    'Print name of first visible component
    Debug.Print vComps(0).Name2

    'Get all edges on the first visible component
    vEdges = swView.GetVisibleEntities(vComps(0), swViewEntityType_Edge)

    'Iterate and select all edges on the first visible component
    Set swSelData = swSelMgr.CreateSelectData
    swSelData.View = swView
    For itr = 0 To UBound(vEdges)
        Set swEnt = vEdges(itr)
        boolstatus = swEnt.Select4(True, swSelData)
    Next itr

End Sub
```

Get All Elements of Sketch Example (VBA)

This example shows how to get all of the elements of a sketch.

```
'-----
' Preconditions:
' 1. Open a part and select a sketch.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets all elements of the sketch.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Enum swSkSegments_e
    swSketchLINE = 0
    swSketchARC = 1
    swSketchELLIPSE = 2
    swSketchSPLINE = 3
    swSketchTEXT = 4
    swSketchPARABOLA = 5
End Enum

Sub ProcessTextFormat(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTextFormat As SldWorks.TextFormat)

    Debug.Print "        BackWards          = " & swTextFormat.BackWards
    Debug.Print "        Bold              = " & swTextFormat.Bold
    Debug.Print "        CharHeight        = " & swTextFormat.CharHeight
    Debug.Print "        CharHeightInPts   = " & swTextFormat.CharHeightInPts
    Debug.Print "        CharSpacingFactor = " & swTextFormat.CharSpacingFactor
    Debug.Print "        Escapement        = " & swTextFormat.Escapement
    Debug.Print "        IsHeightSpecifiedInPts = " & swTextFormat.IsHeightSpecifiedInPts
    Debug.Print "        Italic            = " & swTextFormat.Italic
    Debug.Print "        LineLength        = " & swTextFormat.LineLength
    Debug.Print "        LineSpacing       = " & swTextFormat.LineSpacing
    Debug.Print "        ObliqueAngle      = " & swTextFormat.ObliqueAngle
    Debug.Print "        Strikeout         = " & swTextFormat.Strikeout
    Debug.Print "        TypeFaceName      = " & swTextFormat.TypeFaceName
    Debug.Print "        Underline         = " & swTextFormat.Underline
    Debug.Print "        UpsideDown        = " & swTextFormat.UpsideDown
    Debug.Print "        Vertical          = " & swTextFormat.Vertical
    Debug.Print "        WidthFactor       = " & swTextFormat.WidthFactor
    Debug.Print ""
End Sub

Function TransformSketchPointToModelSpace(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkPt As SldWorks.SketchPoint) As SldWorks.MathPoint

    Dim swMathUtil As SldWorks.MathUtility
    Dim swXform As SldWorks.MathTransform
    Dim nPt(2) As Double
    Dim vPt As Variant
    Dim swMathPt As SldWorks.MathPoint

    nPt(0) = swSkPt.X:      nPt(1) = swSkPt.Y:      nPt(2) = swSkPt.Z
    vPt = nPt

    Set swMathUtil = swApp.GetMathUtility
    Set swXform = swSketch.ModelToSketchTransform
    Set swXform = swXform.Inverse
    Set swMathPt = swMathUtil.CreatePoint((vPt))
    Set swMathPt = swMathPt.MultiplyTransform(swXform)
    Set TransformSketchPointToModelSpace = swMathPt
End Function

Sub ProcessSketchLine(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkLine As SldWorks.SketchLine)

    Dim swStartPt As SldWorks.SketchPoint
    Dim swEndPt As SldWorks.SketchPoint
    Dim swStartModPt As SldWorks.MathPoint
    Dim swEndModPt As SldWorks.MathPoint

    Set swStartPt = swSkLine.GetStartPoint2
    Set swEndPt = swSkLine.GetEndPoint2

    Set swStartModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swStartPt)
    Set swEndModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swEndPt)
    Debug.Print "        Start (sketch)   = (" & swStartPt.X * 1000# & ", " & swStartPt.Y * 1000# & ", " & swStartPt.Z * 1000# & ") mm"
    Debug.Print "        Start (model)    = (" & swStartModPt.ArrayData(0) * 1000# & ", " & swStartModPt.ArrayData(1) * 1000# & ", " & swStartModPt.ArrayData(2) * 1000# & ") mm"
    Debug.Print "        End (sketch)     = (" & swEndPt.X * 1000# & ", " & swEndPt.Y * 1000# & ", " & swEndPt.Z * 1000# & ") mm"
    Debug.Print "        End (model)      = (" & swEndModPt.ArrayData(0) * 1000# & ", " & swEndModPt.ArrayData(1) * 1000# & ", " & swEndModPt.ArrayData(2) * 1000# & ") mm"
End Sub

Sub ProcessSketchArc(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkArc As SldWorks.SketchArc)

    Dim swStartPt As SldWorks.SketchPoint
    Dim swEndPt As SldWorks.SketchPoint
    Dim swCtrPt As SldWorks.SketchPoint
    Dim vNormal As Variant
    Dim swStartModPt As SldWorks.MathPoint
    Dim swEndModPt As SldWorks.MathPoint
    Dim swCtrModPt As SldWorks.MathPoint
```

```

Set swStartPt = swSkArc.GetStartPoint2
Set swEndPt = swSkArc.GetEndPoint2
Set swCtrPt = swSkArc.GetCenterPoint2
Set swStartModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swStartPt)
Set swEndModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swEndPt)
Set swCtrModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swCtrPt)

vNormal = swSkArc.GetNormalVector
Debug.Print "      Start (sketch)  = (" & swStartPt.X * 1000# & ", " & swStartPt.Y * 1000# & ", " & swStartPt.Z * 1000# & ") mm"
Debug.Print "      Start (model)   = (" & swStartModPt.ArrayData(0) * 1000# & ", " & swStartModPt.ArrayData(1) * 1000# & ", " &
swStartModPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "      End (sketch)    = (" & swEndPt.X * 1000# & ", " & swEndPt.Y * 1000# & ", " & swEndPt.Z * 1000# & ") mm"
Debug.Print "      End (model)     = (" & swEndModPt.ArrayData(0) * 1000# & ", " & swEndModPt.ArrayData(1) * 1000# & ", " & swEndModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "      Center(sketch)  = (" & swCtrPt.X * 1000# & ", " & swCtrPt.Y * 1000# & ", " & swCtrPt.Z * 1000# & ") mm"
Debug.Print "      Center(model)   = (" & swCtrModPt.ArrayData(0) * 1000# & ", " & swCtrModPt.ArrayData(1) * 1000# & ", " & swCtrModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "      Radius         = " & swSkArc.GetRadius * 1000# & " mm"
Debug.Print "      IsCircle        = " & CBool(swSkArc.IsCircle)
Debug.Print "      Rot dirn       = " & swSkArc.GetRotationDir

End Sub

Sub ProcessSketchEllipse(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkEllipse As SldWorks.SketchEllipse)

Dim swStartPt As SldWorks.SketchPoint
Dim swEndPt As SldWorks.SketchPoint
Dim swCtrPt As SldWorks.SketchPoint
Dim swMajPt As SldWorks.SketchPoint
Dim swMinPt As SldWorks.SketchPoint
Dim swStartModPt As SldWorks.MathPoint
Dim swEndModPt As SldWorks.MathPoint
Dim swCtrModPt As SldWorks.MathPoint
Dim swMajModPt As SldWorks.MathPoint
Dim swMinModPt As SldWorks.MathPoint

Set swStartPt = swSkEllipse.GetStartPoint2
Set swEndPt = swSkEllipse.GetEndPoint2
Set swCtrPt = swSkEllipse.GetCenterPoint2
Set swMajPt = swSkEllipse.GetMajorPoint2
Set swMinPt = swSkEllipse.GetMinorPoint2
Set swStartModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swStartPt)
Set swEndModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swEndPt)
Set swCtrModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swCtrPt)
Set swMajModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swMajPt)
Set swMinModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swMinPt)

Debug.Print "      Start (sketch)  = (" & swStartPt.X * 1000# & ", " & swStartPt.Y * 1000# & ", " & swStartPt.Z * 1000# & ") mm"
Debug.Print "      Start (model)   = (" & swStartModPt.ArrayData(0) * 1000# & ", " & swStartModPt.ArrayData(1) * 1000# & ", " &
swStartModPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "      End (sketch)    = (" & swEndPt.X * 1000# & ", " & swEndPt.Y * 1000# & ", " & swEndPt.Z * 1000# & ") mm"
Debug.Print "      End (model)     = (" & swEndModPt.ArrayData(0) * 1000# & ", " & swEndModPt.ArrayData(1) * 1000# & ", " & swEndModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "      Center(sketch)  = (" & swCtrPt.X * 1000# & ", " & swCtrPt.Y * 1000# & ", " & swCtrPt.Z * 1000# & ") mm"
Debug.Print "      Center(model)   = (" & swCtrModPt.ArrayData(0) * 1000# & ", " & swCtrModPt.ArrayData(1) * 1000# & ", " & swCtrModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "      Major (sketch)  = (" & swMajPt.X * 1000# & ", " & swMajPt.Y * 1000# & ", " & swMajPt.Z * 1000# & ") mm"
Debug.Print "      Major (model)   = (" & swMajModPt.ArrayData(0) * 1000# & ", " & swMajModPt.ArrayData(1) * 1000# & ", " & swMajModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "      Minor (sketch)  = (" & swMinPt.X * 1000# & ", " & swMinPt.Y * 1000# & ", " & swMinPt.Z * 1000# & ") mm"
Debug.Print "      Minor (model)   = (" & swMinModPt.ArrayData(0) * 1000# & ", " & swMinModPt.ArrayData(1) * 1000# & ", " & swMinModPt.ArrayData(2) *
1000# & ") mm"
End Sub

Sub ProcessSketchSpline(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkSpline As SldWorks.SketchSpline)

Dim vSplinePtArr As Variant
Dim vSplinePt As Variant
Dim swSplinePt As SldWorks.SketchPoint
Dim swSplineModPt As SldWorks.MathPoint

vSplinePtArr = swSkSpline.GetPoints2
For Each vSplinePt In vSplinePtArr
Set swSplinePt = vSplinePt
Set swSplineModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swSplinePt)

Debug.Print "      Spline (sketch)  = (" & swSplinePt.X * 1000# & ", " & swSplinePt.Y * 1000# & ", " & swSplinePt.Z * 1000# & ") mm"
Debug.Print "      Spline (model)   = (" & swSplineModPt.ArrayData(0) * 1000# & ", " & swSplineModPt.ArrayData(1) * 1000# & ", " &
swSplineModPt.ArrayData(2) * 1000# & ") mm"
Next vSplinePt

End Sub

Sub ProcessSketchText(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkText As SldWorks.SketchText)

Dim vCoordPt As Variant
Dim swMathUtil As SldWorks.MathUtility
Dim swXform As SldWorks.MathTransform
Dim swCoordModPt As SldWorks.MathPoint

vCoordPt = swSkText.GetCoordinates
Set swMathUtil = swApp.GetMathUtility
Set swXform = swSketch.ModelToSketchTransform
Set swXform = swXform.Inverse
Set swCoordModPt = swMathUtil.CreatePoint((vCoordPt))
Set swCoordModPt = swCoordModPt.MultiplyTransform(swXform)
Debug.Print "      Coords (sketch)  = (" & vCoordPt(0) * 1000# & ", " & vCoordPt(1) * 1000# & ", " & vCoordPt(2) * 1000# & ") mm"
Debug.Print "      Coords (model)   = (" & swCoordModPt.ArrayData(0) * 1000# & ", " & swCoordModPt.ArrayData(1) * 1000# & ", " &
swCoordModPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "      Use doc fmt      = " & swSkText.GetUseDocTextFormat
Debug.Print "      Text             = " & swSkText.Text

ProcessTextFormat swApp, swModel, swSkText.GetTextFormat

End Sub

```

```
Sub ProcessSketchParabola(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSkParabola As SldWorks.SketchParabola)
```

```

Dim swApexPt As SldWorks.SketchPoint
Dim swStartPt As SldWorks.SketchPoint
Dim swEndPt As SldWorks.SketchPoint
Dim swFocalPt As SldWorks.SketchPoint
Dim swApexModPt As SldWorks.MathPoint
Dim swStartModPt As SldWorks.MathPoint
Dim swEndModPt As SldWorks.MathPoint
Dim swFocalModPt As SldWorks.MathPoint

Set swApexPt = swSkParabola.GetApexPoint2
Set swStartPt = swSkParabola.GetStartPoint2
Set swEndPt = swSkParabola.GetEndPoint2
Set swFocalPt = swSkParabola.GetFocalPoint2
Set swApexModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swApexPt)
Set swStartModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swStartPt)
Set swEndModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swEndPt)
Set swFocalModPt = TransformSketchPointToModelSpace(swApp, swModel, swSketch, swFocalPt)
Debug.Print "    Apex (sketch) = (" & swApexPt.X * 1000# & ", " & swApexPt.Y * 1000# & ", " & swApexPt.Z * 1000# & ") mm"
Debug.Print "    Apex (model) = (" & swApexModPt.ArrayData(0) * 1000# & ", " & swApexModPt.ArrayData(1) * 1000# & ", " &
swApexModPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "    Start (sketch) = (" & swStartPt.X * 1000# & ", " & swStartPt.Y * 1000# & ", " & swStartPt.Z * 1000# & ") mm"
Debug.Print "    Start (model) = (" & swStartModPt.ArrayData(0) * 1000# & ", " & swStartModPt.ArrayData(1) * 1000# & ", " &
swStartModPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "    End (sketch) = (" & swEndPt.X * 1000# & ", " & swEndPt.Y * 1000# & ", " & swEndPt.Z * 1000# & ") mm"
Debug.Print "    End (model) = (" & swEndModPt.ArrayData(0) * 1000# & ", " & swEndModPt.ArrayData(1) * 1000# & ", " & swEndModPt.ArrayData(2) *
1000# & ") mm"
Debug.Print "    Focal (sketch) = (" & swFocalPt.X * 1000# & ", " & swFocalPt.Y * 1000# & ", " & swFocalPt.Z * 1000# & ") mm"
Debug.Print "    Focal (model) = (" & swFocalModPt.ArrayData(0) * 1000# & ", " & swFocalModPt.ArrayData(1) * 1000# & ", " &
swFocalModPt.ArrayData(2) * 1000# & ") mm"

```

```
End Sub
```

```
Sub main()
```

```

Dim sSkSegmentsName(5) As String
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim vSkSegArr As Variant
Dim vSkSeg As Variant
Dim swSkSeg As SldWorks.SketchSegment
Dim swSkLine As SldWorks.SketchLine
Dim swSkArc As SldWorks.SketchArc
Dim swSkEllipse As SldWorks.SketchEllipse
Dim swSkSpline As SldWorks.SketchSpline
Dim swSkText As SldWorks.SketchText
Dim swSkParabola As SldWorks.SketchParabola
Dim vID As Variant
Dim i As Long
Dim bRet As Boolean

```

```

sSkSegmentsName(swSketch.LINE) = "swSketchLINE"
sSkSegmentsName(swSketch.ARC) = "swSketchARC"
sSkSegmentsName(swSketch.ELLIPSE) = "swSketchELLIPSE"
sSkSegmentsName(swSketch.SPLINE) = "swSketchSPLINE"
sSkSegmentsName(swSketch.TEXT) = "swSketchTEXT"
sSkSegmentsName(swSketch.PARABOLA) = "swSketchPARABOLA"

```

```

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFeat = swSelMgr.GetSelectedObject5(1)
Set swSketch = swFeat.GetSpecificFeature
Debug.Print "Feature = " & swFeat.Name & " [" & swSketch.Is3D & "]"
Debug.Print " Sketch Segments:"

```

```

vSkSegArr = swSketch.GetSketchSegments
For Each vSkSeg In vSkSegArr
    Set swSkSeg = vSkSeg
    vID = swSkSeg.GetID
    Debug.Print "    ID = [" & vID(0) & ", " & vID(1) & "]"
    Debug.Print "    Type = " & sSkSegmentsName(swSkSeg.GetType)
    Debug.Print "    ConstGeom = " & swSkSeg.ConstructionGeometry
    Select Case swSkSeg.GetType
        Case swSketch.LINE
            Set swSkLine = swSkSeg
            ProcessSketchLine swApp, swModel, swSketch, swSkLine
        Case swSketch.ARC
            Set swSkArc = swSkSeg
            ProcessSketchArc swApp, swModel, swSketch, swSkArc
        Case swSketch.ELLIPSE
            Set swSkEllipse = swSkSeg
            ProcessSketchEllipse swApp, swModel, swSketch, swSkEllipse
        Case swSketch.SPLINE
            Set swSkSpline = swSkSeg
            ProcessSketchSpline swApp, swModel, swSketch, swSkSpline
        Case swSketch.TEXT
            Set swSkText = swSkSeg
            ProcessSketchText swApp, swModel, swSketch, swSkText
        Case swSketch.PARABOLA
            Set swSkParabola = swSkSeg
            ProcessSketchParabola swApp, swModel, swSketch, swSkParabola
        Case Default
            Debug.Assert False
    End Select

```

```
Next vSkSeg
```

End Sub

Get All Notes in Drawing Template Example (VBA)

This example shows how to get all of the notes in a drawing template.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects all notes in the drawing template.  
' 2. Examine the Immediate window and title block in the drawing.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----  
Option Explicit
```

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swView As SldWorks.View  
    Dim swNote As SldWorks.Note  
    Dim swAnn As SldWorks.Annotation  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swView = swDraw.GetFirstView ' This is the drawing template  
    Set swNote = swView.GetFirstNote2  
  
    swModel.ClearSelection2 (True)  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Do While Not swNote Is Nothing  
        Set swAnn = swNote.GetAnnotation  
        bRet = swAnn.Select2(True, 0)  
        Debug.Print " " & swNote.GetName  
        Debug.Print " " & swNote.GetText  
        Set swNote = swNote.GetNext  
    Loop  
  
End Sub
```

Get All Sheet Metal Feature Data Example (C#)

This example shows how to get all of the sheet metal part's feature data.

```
//-----
// Preconditions:
// 1. Open a sheet metal part.
// 2. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace GetAllSheetMetalFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        Feature swFeat;
        Feature swSubFeat;
        bool bRet;
        int lRet;

        string gaugeTableFile;

        public void Process_CustomBendAllowance(SldWorks swApp, ModelDoc2 swModel, CustomBendAllowance swCustBend)
        {
            Debug.Print("      BendAllowance      = " + swCustBend.BendAllowance * 1000.0 + " mm");
            Debug.Print("      BendDeduction      = " + swCustBend.BendDeduction * 1000.0 + " mm");
            Debug.Print("      BendTableFile      = " + swCustBend.BendTableFile);
            Debug.Print("      KFactor             = " + swCustBend.KFactor);
            Debug.Print("      Type                = " + swCustBend.Type);
        }

        public void Process_SMBaseFlange(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
        {
            Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

            BaseFlangeFeatureData swBaseFlange = default(BaseFlangeFeatureData);
            swBaseFlange = (BaseFlangeFeatureData)swFeat.GetDefinition();

            Debug.Print("      BendRadius = " + swBaseFlange.BendRadius * 1000.0 + " mm");
        }

        public void Process_SheetMetal(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
        {
            Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

            SheetMetalFeatureData swSheetMetal = default(SheetMetalFeatureData);
            swSheetMetal = (SheetMetalFeatureData)swFeat.GetDefinition();

            CustomBendAllowance swCustBend = default(CustomBendAllowance);
            swCustBend = swSheetMetal.GetCustomBendAllowance();
            Debug.Print("      BendRadius = " + swSheetMetal.BendRadius * 1000.0 + " mm");

            Process_CustomBendAllowance(swApp, swModel, swCustBend);

            lRet = swSheetMetal.GetUseGaugeTable(out bRet, out gaugeTableFile);
            Debug.Print("      Use gauge table? " + bRet);
            Debug.Print("      Error code as defined in swSheetMetalModifierError_e: " + lRet);
        }

        public void Process_SM3dBend(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
        {
            Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

            SketchedBendFeatureData swSketchBend = default(SketchedBendFeatureData);
            CustomBendAllowance swCustBend = default(CustomBendAllowance);

            swSketchBend = (SketchedBendFeatureData)swFeat.GetDefinition();
            swCustBend = swSketchBend.GetCustomBendAllowance();

            Debug.Print("      UseDefaultBendAllowance = " + swSketchBend.UseDefaultBendAllowance);
            Debug.Print("      UseDefaultBendRadius = " + swSketchBend.UseDefaultBendRadius);
            Debug.Print("      BendRadius = " + swSketchBend.BendRadius * 1000.0 + " mm");
        }
    }
}
```

```

        Process_CustomBendAllowance(swApp, swModel, swCustBend);
    }

    public void Process_SMMiteredFlange(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        MiterFlangeFeatureData swMiterFlange = default(MiterFlangeFeatureData);
        swMiterFlange = (MiterFlangeFeatureData)swFeat.GetDefinition();

        Debug.Print("    UseDefaultBendAllowance = " + swMiterFlange.UseDefaultBendAllowance);
        Debug.Print("    UseDefaultBendRadius = " + swMiterFlange.UseDefaultBendRadius);
        Debug.Print("    BendRadius = " + swMiterFlange.BendRadius * 1000.0 + " mm");
    }

    public void Process_Bends(SldWorks swApp, ModelDoc2 swModel, BendsFeatureData swBends)
    {
        CustomBendAllowance swCustBend = default(CustomBendAllowance);
        swCustBend = swBends.GetCustomBendAllowance();

        Debug.Print("    BendRadius          = " + swBends.BendRadius * 1000.0 + " mm");
        Debug.Print("    UseDefaultBendAllowance = " + swBends.UseDefaultBendAllowance);
        Debug.Print("    UseDefaultBendRadius   = " + swBends.UseDefaultBendRadius);

        Process_CustomBendAllowance(swApp, swModel, swCustBend);
    }

    public void Process_ProcessBends(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        BendsFeatureData swBends = default(BendsFeatureData);
        swBends = (BendsFeatureData)swFeat.GetDefinition();

        Process_Bends(swApp, swModel, swBends);
    }

    public void Process_FlattenBends(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        BendsFeatureData swBends = default(BendsFeatureData);
        swBends = (BendsFeatureData)swFeat.GetDefinition();

        Process_Bends(swApp, swModel, swBends);
    }

    public void Process_EdgeFlange(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        EdgeFlangeFeatureData swEdgeFlange = default(EdgeFlangeFeatureData);
        swEdgeFlange = (EdgeFlangeFeatureData)swFeat.GetDefinition();

        Debug.Print("    UseDefaultBendRadius = " + swEdgeFlange.UseDefaultBendRadius);
        Debug.Print("    BendRadius = " + swEdgeFlange.BendRadius * 1000.0 + " mm");
    }

    public void Process_FlatPattern(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        FlatPatternFeatureData swFlatPatt = default(FlatPatternFeatureData);
        swFlatPatt = (FlatPatternFeatureData)swFeat.GetDefinition();

        Debug.Print("    Simplify bends? " + swFlatPatt.SimplifyBends);
    }

    public void Process_Hem(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("  +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        HemFeatureData swHem = default(HemFeatureData);
        CustomBendAllowance swCustBend = default(CustomBendAllowance);

        swHem = (HemFeatureData)swFeat.GetDefinition();
        swCustBend = swHem.GetCustomBendAllowance();
    }

```

```

        Debug.Print("    UseDefaultBendAllowance = " + swHem.UseDefaultBendAllowance);
        Debug.Print("    Radius = " + swHem.Radius * 1000.0 + " mm");

        Process_CustomBendAllowance(swApp, swModel, swCustBend);
    }

    public void Process_Jog(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("    +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        JogFeatureData swJog = default(JogFeatureData);
        CustomBendAllowance swCustBend = default(CustomBendAllowance);

        swJog = (JogFeatureData)swFeat.GetDefinition();
        swCustBend = swJog.GetCustomBendAllowance();

        Debug.Print("    UseDefaultBendAllowance = " + swJog.UseDefaultBendAllowance);
        Debug.Print("    UseDefaultBendRadius = " + swJog.UseDefaultBendRadius);
        Debug.Print("    BendRadius = " + swJog.BendRadius * 1000.0 + " mm");

        Process_CustomBendAllowance(swApp, swModel, swCustBend);
    }

    public void Process_LoftedBend(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("    +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        LoftedBendsFeatureData swLoftBend = default(LoftedBendsFeatureData);
        swLoftBend = (LoftedBendsFeatureData)swFeat.GetDefinition();
    }

    public void Process_Rip(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("    +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        RipFeatureData swRip = default(RipFeatureData);
        swRip = (RipFeatureData)swFeat.GetDefinition();
    }

    public void Process_CornerFeat(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("    +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        ClosedCornerFeatureData swCloseCorner = default(ClosedCornerFeatureData);
        swCloseCorner = (ClosedCornerFeatureData)swFeat.GetDefinition();
    }

    public void Process_OneBend(SldWorks swApp, ModelDoc2 swModel, Feature swFeat)
    {
        Debug.Print("    +" + swFeat.Name + " [" + swFeat.GetTypeName() + "]");

        OneBendFeatureData swOneBend = default(OneBendFeatureData);
        CustomBendAllowance swCustBend = default(CustomBendAllowance);

        swOneBend = (OneBendFeatureData)swFeat.GetDefinition();
        swCustBend = swOneBend.GetCustomBendAllowance();

        Debug.Print("    UseDefaultBendAllowance = " + swOneBend.UseDefaultBendAllowance);
        Debug.Print("    UseDefaultBendRadius = " + swOneBend.UseDefaultBendRadius);

        Process_CustomBendAllowance(swApp, swModel, swCustBend);
    }

    public void Main()
    {
        swModel = (ModelDoc2)swApp.ActiveDoc;
        swSelMgr = (SelectionMgr)swModel.SelectionManager;
        swFeat = (Feature)swModel.FirstFeature();

        Debug.Print("File = " + swModel.GetPathName());

        while ((swFeat != null))
        {
            // Process top-level sheet metal features
            switch (swFeat.GetTypeName())

```

```

{
    case "SMBaseFlange":
        Process_SMBaseFlange(swApp, swModel, swFeat);

        break;
    case "SheetMetal":
        Process_SheetMetal(swApp, swModel, swFeat);

        break;
    case "SM3dBend":
        Process_SM3dBend(swApp, swModel, swFeat);

        break;
    case "SMMiteredFlange":
        Process_SMMiteredFlange(swApp, swModel, swFeat);

        break;
    case "ProcessBends":
        Process_ProcessBends(swApp, swModel, swFeat);

        break;
    case "FlattenBends":
        Process_FlattenBends(swApp, swModel, swFeat);

        break;
    case "EdgeFlange":
        Process_EdgeFlange(swApp, swModel, swFeat);

        break;
    case "FlatPattern":
        Process_FlatPattern(swApp, swModel, swFeat);

        break;
    case "Hem":
        Process_Hem(swApp, swModel, swFeat);

        break;
    case "Jog":
        Process_Jog(swApp, swModel, swFeat);

        break;
    case "LoftedBend":
        Process_LoftedBend(swApp, swModel, swFeat);

        break;
    case "Rip":
        Process_Rip(swApp, swModel, swFeat);

        break;
    case "CornerFeat":
        Process_CornerFeat(swApp, swModel, swFeat);

        break;
    default:
        break;
    // Probably not a sheet metal feature
}

// process sheet metal sub-features
swSubFeat = (Feature)swFeat.GetFirstSubFeature();

while ((swSubFeat != null))
{
    switch (swSubFeat.GetTypeName())
    {
        case "OneBend":
            Process_OneBend(swApp, swModel, swSubFeat);

            break;
        default:
            break;
        // Probably not a sheet metal feature
    }

    swSubFeat = (Feature)swSubFeat.GetNextSubFeature();
}

swFeat = (Feature)swFeat.GetNextFeature();
}

}

public SldWorks swApp;
}
}

```

Get All Sheet Metal Feature Data Example (VBA)

This example shows how to get all of the sheet metal part's feature data.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
Dim swApp As SldWorks.SldWorks
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swSubFeat As SldWorks.Feature
Dim bRet As Boolean
Dim lRet As Long
Dim gaugeTableFile As String

Option Explicit

Sub Process_CustomBendAllowance _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swCustBend As SldWorks.CustomBendAllowance _
)
    Debug.Print "    BendAllowance    = " & swCustBend.BendAllowance * 1000# & " mm"
    Debug.Print "    BendDeduction    = " & swCustBend.BendDeduction * 1000# & " mm"
    Debug.Print "    BendTableFile    = " & swCustBend.BendTableFile
    Debug.Print "    KFactor          = " & swCustBend.KFactor
    Debug.Print "    Type              = " & swCustBend.Type
End Sub

Sub Process_SMBaseFlange _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)
    Debug.Print "    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"
    Dim swBaseFlange As SldWorks.BaseFlangeFeatureData
    Set swBaseFlange = swFeat.GetDefinition
    Debug.Print "    BendRadius = " & swBaseFlange.BendRadius * 1000# & " mm"
End Sub

Sub Process_SheetMetal _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)
    Debug.Print "    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"
    Dim swSheetMetal As SldWorks.SheetMetalFeatureData
    Set swSheetMetal = swFeat.GetDefinition

    Dim swCustBend As SldWorks.CustomBendAllowance
    Set swCustBend = swSheetMetal.GetCustomBendAllowance
    Debug.Print "    BendRadius = " & swSheetMetal.BendRadius * 1000# & " mm"
    Process_CustomBendAllowance swApp, swModel, swCustBend

    lRet = swSheetMetal.GetUseGaugeTable(bRet, gaugeTableFile)
    Debug.Print "    Use gauge table? " & bRet
    Debug.Print "    Error code as defined in swSheetMetalModifierError_e: " & lRet
End Sub

Sub Process_SM3dBend _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)
    Debug.Print "    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"
    Dim swSketchBend As SldWorks.SketchedBendFeatureData
    Dim swCustBend As SldWorks.CustomBendAllowance
    Set swSketchBend = swFeat.GetDefinition
    Set swCustBend = swSketchBend.GetCustomBendAllowance
    Debug.Print "    UseDefaultBendAllowance = " & swSketchBend.UseDefaultBendAllowance
    Debug.Print "    UseDefaultBendRadius = " & swSketchBend.UseDefaultBendRadius
    Debug.Print "    BendRadius = " & swSketchBend.BendRadius * 1000# & " mm"
    Process_CustomBendAllowance swApp, swModel, swCustBend
End Sub
```

```

Sub Process_SMMiteredFlange _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swMiterFlange As SldWorks.MiterFlangeFeatureData
    Set swMiterFlange = swFeat.GetDefinition

    Debug.Print "    UseDefaultBendAllowance = " & swMiterFlange.UseDefaultBendAllowance
    Debug.Print "    UseDefaultBendRadius = " & swMiterFlange.UseDefaultBendRadius
    Debug.Print "    BendRadius = " & swMiterFlange.BendRadius * 1000# & " mm"

End Sub

Sub Process_Bends _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swBends As SldWorks.BendsFeatureData _
)

    Dim swCustBend As SldWorks.CustomBendAllowance
    Set swCustBend = swBends.GetCustomBendAllowance

    Debug.Print "    BendRadius = " & swBends.BendRadius * 1000# & " mm"
    Debug.Print "    UseDefaultBendAllowance = " & swBends.UseDefaultBendAllowance
    Debug.Print "    UseDefaultBendRadius = " & swBends.UseDefaultBendRadius

    Process_CustomBendAllowance swApp, swModel, swCustBend

End Sub

Sub Process_ProcessBends _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swBends As SldWorks.BendsFeatureData
    Set swBends = swFeat.GetDefinition

    Process_Bends swApp, swModel, swBends

End Sub

Sub Process_FlattenBends _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swBends As SldWorks.BendsFeatureData
    Set swBends = swFeat.GetDefinition

    Process_Bends swApp, swModel, swBends

End Sub

Sub Process_EdgeFlange _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swEdgeFlange As SldWorks.EdgeFlangeFeatureData
    Set swEdgeFlange = swFeat.GetDefinition

    Debug.Print "    UseDefaultBendRadius = " & swEdgeFlange.UseDefaultBendRadius
    Debug.Print "    BendRadius = " & swEdgeFlange.BendRadius * 1000# & " mm"

End Sub

Sub Process_FlatPattern _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swFlatPatt As SldWorks.FlatPatternFeatureData
    Set swFlatPatt = swFeat.GetDefinition

    Debug.Print "    Simplify bends? " & swFlatPatt.SimplifyBends

End Sub

Sub Process_Hem _
(
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

```

```

Dim swHem As SldWorks.HemFeatureData
Dim swCustBend As SldWorks.CustomBendAllowance
Set swHem = swFeat.GetDefinition
Set swCustBend = swHem.GetCustomBendAllowance

Debug.Print " UseDefaultBendAllowance = " & swHem.UseDefaultBendAllowance
Debug.Print " Radius = " & swHem.Radius * 1000# & " mm"

Process_CustomBendAllowance swApp, swModel, swCustBend
End Sub

Sub Process_Jog _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swJog As SldWorks.JogFeatureData
    Dim swCustBend As SldWorks.CustomBendAllowance

    Set swJog = swFeat.GetDefinition
    Set swCustBend = swJog.GetCustomBendAllowance

    Debug.Print " UseDefaultBendAllowance = " & swJog.UseDefaultBendAllowance
    Debug.Print " UseDefaultBendRadius = " & swJog.UseDefaultBendRadius
    Debug.Print " BendRadius = " & swJog.BendRadius * 1000# & " mm"

    Process_CustomBendAllowance swApp, swModel, swCustBend
End Sub

Sub Process_LoftedBend _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swLoftBend As SldWorks.LoftedBendsFeatureData
    Set swLoftBend = swFeat.GetDefinition

End Sub

Sub Process_Rip _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swRip As SldWorks.RipFeatureData
    Set swRip = swFeat.GetDefinition

End Sub

Sub Process_CornerFeat _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swCloseCorner As SldWorks.ClosedCornerFeatureData
    Set swCloseCorner = swFeat.GetDefinition

End Sub

Sub Process_OneBend _
( _
    swApp As SldWorks.SldWorks, _
    swModel As SldWorks.ModelDoc2, _
    swFeat As SldWorks.Feature _
)

    Debug.Print " +" & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    Dim swOneBend As SldWorks.OneBendFeatureData
    Dim swCustBend As SldWorks.CustomBendAllowance

    Set swOneBend = swFeat.GetDefinition
    Set swCustBend = swOneBend.GetCustomBendAllowance

    Debug.Print " UseDefaultBendAllowance = " & swOneBend.UseDefaultBendAllowance
    Debug.Print " UseDefaultBendRadius = " & swOneBend.UseDefaultBendRadius

    Process_CustomBendAllowance swApp, swModel, swCustBend
End Sub

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

```

```

Set swSelMgr = swModel.SelectionManager
Set swFeat = swModel.FirstFeature
Debug.Print "File = " & swModel.GetPathName
Do While Not swFeat Is Nothing
    ' Process top-level sheet metal features
    Select Case swFeat.GetType
        Case "SMBaseFlange"
            Process_SMBaseFlange swApp, swModel, swFeat
        Case "SheetMetal"
            Process_SheetMetal swApp, swModel, swFeat
        Case "SM3dBend"
            Process_SM3dBend swApp, swModel, swFeat
        Case "SMMiteredFlange"
            Process_SMMiteredFlange swApp, swModel, swFeat
        Case "ProcessBends"
            Process_ProcessBends swApp, swModel, swFeat
        Case "FlattenBends"
            Process_FlattenBends swApp, swModel, swFeat
        Case "EdgeFlange"
            Process_EdgeFlange swApp, swModel, swFeat
        Case "FlatPattern"
            Process_FlatPattern swApp, swModel, swFeat
        Case "Hem"
            Process_Hem swApp, swModel, swFeat
        Case "Jog"
            Process_Jog swApp, swModel, swFeat
        Case "LoftedBend"
            Process_LoftedBend swApp, swModel, swFeat
        Case "Rip"
            Process_Rip swApp, swModel, swFeat
        Case "CornerFeat"
            Process_CornerFeat swApp, swModel, swFeat
        Case Else
            ' Probably not a sheet metal feature
    End Select
    ' process sheet metal sub-features
    Set swSubFeat = swFeat.GetFirstSubFeature
    Do While Not swSubFeat Is Nothing
        Select Case swSubFeat.GetType
            Case "OneBend"
                Process_OneBend swApp, swModel, swSubFeat
            Case Else
                ' Probably not a sheet metal feature
        End Select
        Set swSubFeat = swSubFeat.GetNextSubFeature()
    Loop
    Set swFeat = swFeat.GetNextFeature
Loop
End Sub

```